

FRE7241 Algorithmic Portfolio Management

Lecture#4, Fall 2025

Jerzy Pawlowski jp3900@nyu.edu

NYU Tandon School of Engineering

November 18, 2025



NYU

**TANDON SCHOOL
OF ENGINEERING**

Interest Rate Yield Curve and Stock Returns

The level of interest rates and the shape of the yield curve can be used to forecast stock returns.

The *FRED* database provides historical constant maturity Treasury yields:

<https://fred.stlouisfed.org/series/DGS5>

```
> # Download time series of bond yields
> # symbolv <- c("DGS1", "DGS2", "DGS5", "DGS10", "DGS20", "DGS30")
> # ratesenv <- new.env()
> # quantmod::getSymbols(symbolv, env=ratesenv, src="FRED")
> # Load constant maturity Treasury rates
> load(file="/Users/jerzy/Develop/lecture_slides/data/rates_data.RData")
> # Combine rates into single xts series
> ratev <- do.call(cbind, as.list(ratesenv))
> # Sort the columns of rates according bond maturity
> namev <- colnames(ratev)
> namev <- substr(namev, start=4, stop=10)
> namev <- as.numeric(namev)
> indeks <- order(namev)
> ratev <- ratev[, indeks]
> # Align rates dates with VTI prices
> closep <- log(quantmod::Cl(rutils::etfenv$VTI))
> colnames(closep) <- "VTI"
> datev <- zoo::index(closep)
> ratev <- na.omit(ratev[datev])
> closep <- closep[zoo::index(ratev)]
> datev <- zoo::index(closep)
> nrow <- NROW(closep)
```

```
> # Calculate VTI returns and IR changes
> retp <- rutils::diffit(closep)
> retr <- rutils::diffit(ratev)
> # Regress VTI returns versus the lagged rate differences
> predm <- rutils::lagit(retr)
> regmod <- lm(retp ~ predm)
> summary(regmod)
> # Regress VTI returns before and after 2010
> summary(lm(retp["/2010"] ~ predm["/2010"]))
> summary(lm(retp["2010/"] ~ predm["2010/"]))
```

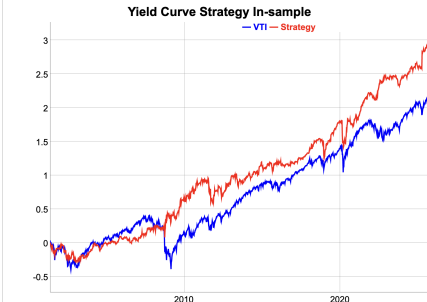
Yield Curve Strategy In-Sample

For in-sample forecasts, the training set and the test set are the same. The model is calibrated on the data that is used for forecasting.

Although it's not realistic to achieve the in-sample performance, it's useful because it provides insights into how the model works.

The in-sample strategy performs well in periods of high volatility, but not as well in periods of low volatility.

```
> # Define predictor with intercept term
> predm <- rutils::lagit(retr)
> predm <- cbind(rep(1, NROW(predm)), predm)
> colnames(predm)[1] <- "intercept"
> # Calculate inverse of predictor
> invreg <- MASS::ginv(predm)
> # Calculate coefficients from response and inverse of predictor
> respv <- retp
> coeff <- drop(invreg %*% respv)
> # Calculate forecasts and PnLs in-sample
> fcasts <- (predm %*% coeff)
> fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
> # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
> pnls <- fcasts*respv
> # Calculate the in-sample factors
> factv <- lapply(1:NCOL(predm), function(x) predm[, x]*coeff[x])
> factv <- do.call(cbind, factv)
> # Or:
> # foo <- HighFreq::mult_mat(coeff, predm)
> # all.equal(foo, coredata(factv), check.attributes=FALSE)
> # Calculate the factor volatilities
> apply(factv, 2, sd)
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv)/sd(pnls)
```

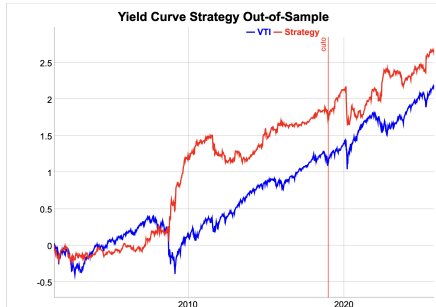


```
> # Plot dygraph of in-sample YC strategy
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> cor(wealthv)
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Yield Curve Strategy In-sample") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Yield Curve Strategy Out-of-Sample

For out-of-sample forecasts, the training set and the test set are separate. The model is calibrated on the training set, and forecasts are calculated using the test set.

```
> # Calculate inverse of predictor in-sample
> invreg <- MASS::ginv(predm["/2018"])
> # Calculate coefficients in-sample
> coeff <- drop(invreg %*% respv["/2018"])
> # Calculate forecasts and PnLs
> fcasts <- (predm %*% coeff)
> # fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
> # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
> pnls <- fcasts*respv
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv)/sd(pnls)
```



```
> # Plot dygraph of out-of-sample YC strategy
> wealthv <- cbind(retp, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Yield Curve Strategy Out-of-Sample") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(end(predm["/2018"]), label="cutoff", strokePattern="solid",
+   dyLegend(show="always", width=300)
```

Rolling Yearly Yield Curve Strategy

In the rolling yearly yield curve strategy, the model is recalibrated at the end of every year using a training set of data from the past lookback days. The coefficients are applied to calculate out-of-sample forecasts in the following year.

```
> # Define yearly dates
> endd <- rutils::calc_endpoints(respv, interval="years")
> # endd <- index(closep)[endd]
> # Perform loop over yearly dates
> library(parallel) ## Load package parallel
> ncores <- detectCores() - 1
> lookb <- 500
> fcasts <- mclapply(seq_along(endd)[-1], function(tday) {
+   ## Define in-sample and out-of-sample intervals
+   insample <- (max(1, endd[tday-1]-lookb):endd[tday-1])
+   ## insample <- (1:endd[tday-1]) ## Expanding look-back
+   outsample <- (endd[tday-1]+1):endd[tday]
+   ## Calculate coefficients in-sample
+   invreg <- MASS::ginv(predm[insample, ])
+   coeff <- drop(invreg %%% respv[insample, ])
+   ## Calculate forecasts out-of-sample
+   fcasts <- (predm[outsample, ] %%% coeff)
+ }, mc.cores=ncores) ## end mclapply
> fcasts <- do.call(rbind, fcasts)
> # fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
> # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
> pnls <- fcasts*respv
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv)/sd(pnls)
```



```
> # Plot dygraph of rolling yearly YC strategy
> wealthv <- cbind(respv, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Yearly Yield Curve Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

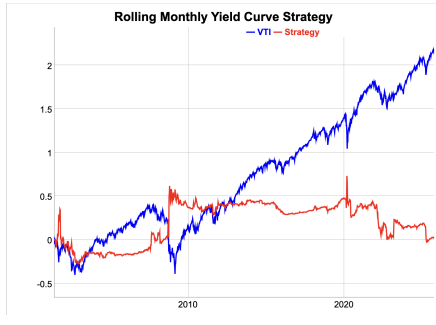
Rolling Monthly Yield Curve Strategy

In the rolling monthly yield curve strategy, the model is recalibrated at the end of every month using a training set of the past lookback days. The coefficients are applied to perform out-of-sample forecasts in the following month.

Research shows that looking back roughly a year provides the best out-of-sample forecasts.

The rolling monthly strategy performs better than the yearly strategy, but mostly in periods of high volatility, and otherwise it's flat.

```
> # Define monthly dates
> endd <- rutils::calc_endpoints(respv, interval="month")
> fcasts <- mclapply(seq_along(endd)[-1], function(tday) {
+   ## Define in-sample and out-of-sample intervals
+   insample <- (max(1, endd[tday-1]-lookb):endd[tday-1])
+   ## insample <- (1:endd[tday-1]) ## Expanding look-back
+   outsample <- (endd[tday-1]+1):endd[tday]
+   ## Calculate coefficients in-sample
+   invreg <- MASS::ginv(predm[insample, ])
+   coeff <- drop(invreg %*% respv[insample, ])
+   ## Calculate forecasts out-of-sample
+   fcasts <- (predm[outsample, ] %*% coeff)
+ }, mc.cores=ncores) ## end mclapply
> fcasts <- do.call(rbind, fcasts)
> # fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
> # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
> pnls <- fcasts*respv
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv)/sd(pnls)
```



```
> # Plot dygraph of rolling monthly YC strategy
> wealthv <- cbind(respv, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Monthly Yield Curve Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Rolling Weekly Yield Curve Strategy

In the rolling weekly yield curve strategy, the model is recalibrated at the end of every week using a training set of the past lookback days. The coefficients are applied to perform out-of-sample forecasts in the following week.

The rolling weekly strategy performs worse than the monthly strategy, because the coefficients have a larger variance and estimation error.

```
> # Define weekly dates
> endd <- rutils::calc_endpoints(respv, interval="weeks")
> fcasts <- mclapply(seq_along(endd)[-1], function(tday) {
+   ## Define in-sample and out-of-sample intervals
+   insample <- (max(1, endd[tday-1]-lookb):endd[tday-1])
+   ## insample <- (1:endd[tday-1]) ## Expanding look-back
+   outsample <- (endd[tday-1]+1):endd[tday]
+   ## Calculate coefficients in-sample
+   invreg <- MASS::ginv(predm[insample, ])
+   coeff <- drop(invreg %*% respv[insample, ])
+   ## Calculate forecasts out-of-sample
+   fcasts <- (predm[outsample, ] %*% coeff)
+ }, mc.cores=ncores) ## end mclapply
> fcasts <- do.call(rbind, fcasts)
> # fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
> # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
> pnls <- fcasts*respv
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv)/sd(pnls)
```



```
> # Plot dygraph of rolling weekly YC strategy
> wealthv <- cbind(respv, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Weekly Yield Curve Strategy") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Yield Curve Strategy With Dimension Reduction In-Sample

The technique of *dimension reduction* is used to reduce the number of predictors in a model, for example in portfolio optimization.

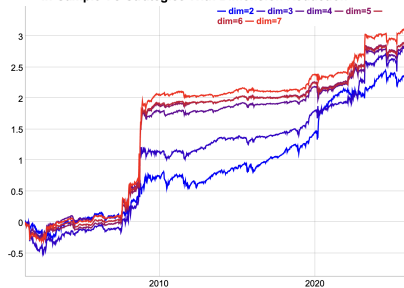
Regularization of the inverse predictor matrix improves the in-sample performance of the yield curve strategy.

The best performing in-sample YC strategy are with the intermediate order parameters $\text{dimax} = 5, 6$.

Although it's not realistic to achieve the in-sample performance, it's useful because it provides insights into how the model can be improved.

```
> # Calculate in-sample pnls for different dimax values
> dimv <- 2:7
> pnls <- mclapply(dimv, function(dimax) {
+   invred <- HighFreq::calc_invsvd(predm, dimax=dimax)
+   coeff <- drop(invred %*% respv)
+   fcasts <- (predm %*% coeff)
+   # fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
+   # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
+   pnls <- fcasts*respv
+   pnls/sd(pnls)
+ }, mc.cores=ncores) ## end mclapply
> pnls <- sd(respv)*do.call(cbind, pnls)
> colnames(pnls) <- paste0("dim=", dimv)
```

In-Sample YC Strategies With Dimension Reduction



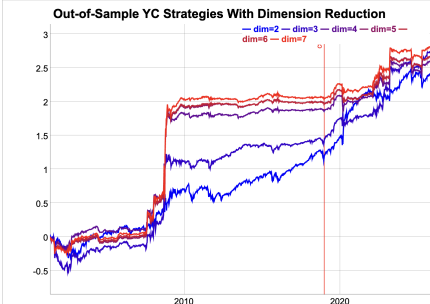
```
> # Plot dygraph of in-sample pnls
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls, interval="weeks")
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="In-Sample YC Strategies With Dimension Reduction") %>%
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```


Yield Curve Strategy With Dimension Reduction Out-of-Sample

For out-of-sample forecasts, the training set and the test set are separate. The model is calibrated on the training set, and forecasts are calculated using the test set.

The best performing out-of-sample YC strategy is with the smallest order parameter (strongest dimension reduction) $dimax = 2$.

```
> # Calculate in-sample pnls for different dimax values
> pnls <- mclapply(dimv, function(dimax) {
+   invred <- HighFreq::calc_invsvd(predm["/2018"], dimax=dimax)
+   coeff <- drop(invred %*% respv["/2018"])
+   fcasts <- (predm %*% coeff)
+   # fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
+   # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
+   pnls <- fcasts*respv
+   pnls/sd(pnls)
+ }, mc.cores=ncores) ## end mclapply
> pnls <- sd(respv)*do.call(cbind, pnls)
> colnames(pnls) <- paste0("dim=", dimv)
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(pnls["/2018/"], function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```



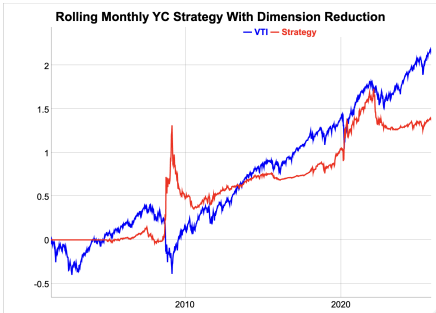
```
> # Plot dygraph of out-of-sample pnls
> colorv <- colorRampPalette(c("blue", "red"))(NCOL(pnls))
> endw <- rutils::calc_endpoints(pnls, interval="weeks")
> dygraphs::dygraph(cumsum(pnls)[endw],
+   main="Out-of-Sample YC Strategies With Dimension Reduction") %>%
+   dyOptions(colors=colorv, strokeWidth=2) %>%
+   dyEvent(end(predm["/2018"]), label="cutoff", strokePattern="solid")
+   dyLegend(show="always", width=300)
```

Rolling Monthly Yield Curve Strategy With Dimension Reduction

The rolling monthly strategy with dimension reduction $\text{dimax} = 2$ performs better than the standard strategy.

In the rolling monthly yield curve strategy, the model is recalibrated at the end of every month using a training set of the past lookb days. The coefficients are applied to perform out-of-sample forecasts in the following month.

```
> # Define monthly dates
> endd <- rutils::calc_endpoints(respv, interval="month")
> enddd <- seq_along(endd)[endd > lookb]
> # Perform loop over monthly dates
> lookb <- 500
> dimax <- 2
> fcsts <- mclapply(enddd, function(tday) {
+   ## Define in-sample and out-of-sample intervals
+   insample <- (max(1, endd[tday-1]-lookb):endd[tday-1])
+   outsample <- (endd[tday-1]+1):endd[tday]
+   ## Calculate coefficients in-sample
+   invreg <- HighFreq::calc_invsvd(predm[insample, ], dimax=dimax)
+   coeff <- drop(invreg %*% respv[insample, ])
+   ## Calculate forecasts out-of-sample
+   fcsts <- (predm[outsample, ] %*% coeff)
+ }, mc.cores=ncores) ## end mclapply
> fcsts <- do.call(rbind, fcsts)
> fcsts <- rbind(matrix(rep(0, nrow=NCOL(fcsts)), nc=1), fcsts)
> # fcastv <- sqrt(HighFreq::run_var(fcsts, lambda=0.4)[, 2])
> # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcsts/fcastv, 0)
> pnls <- fcsts*respv
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv)/sd(pnls)
```



```
> # Plot dygraph of rolling monthly YC strategy
> wealthv <- cbind(respv, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Monthly YC Strategy With Dimension Reduction") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Rolling Weekly Yield Curve Strategy With Shrinkage

In the rolling weekly yield curve strategy, the model is recalibrated at the end of every week using a training set of the past lookb days. The coefficients are applied to perform out-of-sample forecasts in the following week.

```
> # Define weekly dates
> endd <- rutils::calc_endpoints(closep, interval="weeks")
> enddd <- seq_along(endd)[endd > lookb]
> fcsts <- mclapply(enddd, function(tday) {
+   ## Define in-sample and out-of-sample intervals
+   insample <- (max(1, endd[tday-1]-lookb):endd[tday-1])
+   outsample <- (endd[tday-1]+1):endd[tday]
+   ## Calculate coefficients in-sample
+   invreg <- HighFreq::calc_invsd(predm[insample, ], dimax=dimax)
+   coeff <- drop(invreg %*% respv[insample, ])
+   ## Calculate forecasts out-of-sample
+   fcsts <- (predm[outsample, ] %*% coeff)
+ }, mc.cores=ncores) ## end mclapply
> fcsts <- do.call(rbind, fcsts)
> fcsts <- rbind(matrix(rep(0, nrow=NROW(fcsts)), nc=1), fcsts)
> # fcstcv <- sqrt(HighFreq::run_var(fcsts, lambda=0.4)[, 2])
> # fcsts <- ifelse(fcstcv > mad(fcstcv)/20, fcsts/fcstcv, 0)
> pnls <- fcsts*respv
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv)/sd(pnls)
```



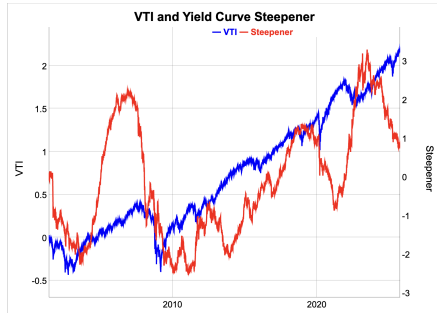
```
> # Plot dygraph of rolling weekly YC strategy
> wealthv <- cbind(respv, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Rolling Weekly YC Strategy With Dimension Reduction") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Yield Curve Principal Components and Stock Returns

The principal components of the interest rate yield curve can also be used as predictors of stock indices.

The second principal component describes the steepening and flattening of the yield curve, and it's an indicator of investor risk appetite. So it's also related to bullish and bearish market periods.

```
> # Calculate PCA of rates from correlation matrix
> eigend <- eigen(cor(retr))
> pcar <- (retr %*% eigend$vectors)
> colnames(pcar) <- paste0("PC", 1:6)
> pcar <- xts::xts(pcar, datev)
> # Define predictor as the YC PCAs
> predm <- rutils::lagit(pcar)
> regmod <- lm(retsp ~ predm)
> summary(regmod)
> # After 2010, the PCAs are not good predictors
> regmod <- lm(retsp["2010/"] ~ predm["2010/"])
> summary(regmod)
```



```
> # Plot YC steepener principal component with VTI
> datav <- cbind(respv, pcar[, 2])
> colnames(datav) <- c("VTI", "Steepener")
> colv <- colnames(datav)
> dygraphs::dygraph(cumsum(datav),
+   main="VTI and Yield Curve Steepener") %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=2, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=2, col="red") %>%
+   dyLegend(show="always", width=300)
```

PCA Yield Curve Strategy In-Sample

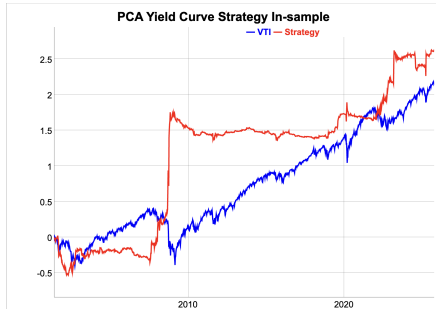
The best performing PCA yield curve strategy uses only the first two principal components of the yield curve as predictors of stock returns.

For in-sample forecasts, the training set and the test set are the same. The model is calibrated on the data that is used for forecasting.

Although it's not realistic to achieve the in-sample performance, it's useful because it provides insights into how the model works.

The in-sample strategy performs well in periods of high volatility, but otherwise it's flat.

```
> # Define predictor without intercept term
> predm <- rutils::lagit(pcar[, 1:2])
> # Calculate inverse of predictor
> invreg <- MASS::ginv(predm)
> # Calculate coefficients from response and inverse of predictor
> coeff <- drop(invreg %%% respv)
> # Calculate forecasts and PnLs in-sample
> fcasts <- (predm %%% coeff)
> # fcastv <- sqrt(HighFreq::run_var(fcasts, lambda=0.4)[, 2])
> # fcasts <- ifelse(fcastv > mad(fcastv)/20, fcasts/fcastv, 0)
> pnls <- fcasts*respv
> # Scale the PnL volatility to that of VTI
> pnls <- pnls*sd(respv[respv<0])/sd(pnls[pnls<0])
> # Calculate in-sample factors
> factv <- (predm*coeff)
> apply(factv, 2, sd)
```



```
> # Plot dygraph of in-sample YC strategy
> wealthv <- cbind(respv, pnls)
> colnames(wealthv) <- c("VTI", "Strategy")
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> colv <- colnames(wealthv)
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="PCA Yield Curve Strategy In-sample") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Calculating the Trailing Variance of Asset Returns

The variance of asset returns exhibits *heteroskedasticity*, i.e. it changes over time.

The trailing variance of returns is given by:

$$\sigma_t^2 = \frac{1}{k-1} \sum_{j=0}^{k-1} (r_{t-j} - \bar{r}_t)^2$$

$$\bar{r}_t = \frac{1}{k} \sum_{j=0}^{k-1} r_{t-j}$$

Where k is the *look-back interval* equal to the number of data points for performing aggregations over the past.

It's also possible to calculate the trailing variance in R using vectorized functions, without using an `apply()` loop.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> nrow <- NROW(retp)
> # Define end points
> endd <- 1:NROW(retp)
> # Start points are multi-period lag of endd
> lookb <- 11
> startp <- c(rep_len(0, lookb-1), endd[1:(nrow-lookb+1)])
> # Calculate trailing variance in sapply() loop - takes long
> varv <- sapply(1:nrow, function(it) {
+   retp <- retp[startp[it]:endd[it]]
+   sum((retp - mean(retp))^2)/lookb
+ }) ## end sapply
> # Use only vectorized functions
> retc <- cumsum(retp)
> retc <- (retc - c(rep_len(0, lookb), retc[1:(nrow-lookb)]))
> retc2 <- cumsum(retp^2)
> retc2 <- (retc2 - c(rep_len(0, lookb), retc2[1:(nrow-lookb)]))
> var2 <- (retc2 - retc^2/lookb)/lookb
> all.equal(varv[-(1:lookb)], as.numeric(var2)[-(1:lookb)])
> # Or using package rutils
> retc <- rutils::roll_sum(retp, lookb=lookb)
> retc2 <- rutils::roll_sum(retp^2, lookb=lookb)
> var2 <- (retc2 - retc^2/lookb)/lookb
> # Coerce variance into xts
> tail(varv)
> class(varv)
> varv <- xts(varv, order.by=zoo::index(retp))
> colnames(varv) <- "VTI.variance"
> head(varv)
```

Calculating the Trailing Variance Using Package *roll*

The package *roll* contains functions for calculating *weighted* trailing aggregations over *vectors* and *time series* objects:

- `roll_sum()` for the *weighted* trailing sum,
- `roll_var()` for the *weighted* trailing variance,
- `roll_scale()` for the trailing scaling and centering of time series,
- `roll_pcr()` for the trailing principal component regressions of time series.

The *roll* functions are about 1,000 times faster than `apply()` loops!

The *roll* functions are extremely fast because they perform calculations in *parallel* in compiled C++ code, using packages *Rcpp*, *RcppArmadillo*, and *RcppParallel*.

The *roll* functions accept *xts* time series, and they return *xts*.

```
> # Calculate trailing VTI variance using package HighFreq
> varv <- roll::roll_var(retp, width=lookb)
> colnames(varv) <- "Variance"
> head(varv)
> sum(is.na(varv))
> varv[1:(lookb-1)] <- 0
> # Benchmark calculation of trailing variance
> library(microbenchmark)
> summary(microbenchmark(
+   sapply=sapply(1:nrows, function(it) {
+     var(retp[startp[it]:endp[it]])
+   }),
+   roll=roll::roll_var(retp, width=lookb),
+   times=10))[, c(1, 4, 5)]
```

Trailing *EMA* Realized Volatility Estimator

Time-varying volatility can be more accurately estimated using an *Exponentially Weighted Moving Average (EMA)* variance estimator.

If the *time series* has zero *expected* mean, then the *EMA realized variance estimator* can be written approxiamtely as:

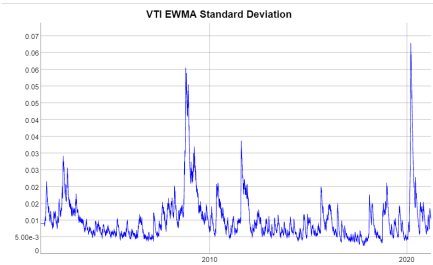
$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) r_t^2 = (1 - \lambda) \sum_{j=0}^{\infty} \lambda^j r_{t-j}^2$$

σ_t^2 is the weighted *realized* variance, equal to the weighted average of the point realized variance for period *i* and the past *realized* variance.

The parameter λ determines the rate of decay of the *EMA* weights, with smaller values of λ producing faster decay, giving more weight to recent realized variance, and vice versa.

The function `stats::C_cfilter()` calculates the convolution of a vector or a time series with a filter of coefficients (weights).

The function `stats::C_cfilter()` is very fast because it's compiled C++ code.



```
> # Calculate EMA VTI variance using compiled C++ function
> lookb <- 51
> weightv <- exp(-0.1*1:lookb)
> weightv <- weightv/sum(weightv)
> varv <- .Call(stats::C_cfilter, retv^2, filter=weightv, sides=1,
> varv[1:(lookb-1)] <- varv[lookb]
> # Plot EMA volatility
> varv <- xts::xts(sqrt(varv), order.by=zoo::index(retv))
> dygraphs::dygraph(varv, main="VTI EMA Volatility") %>%
+   dyOptions(colors="blue") %>% dyLegend(show="always", width=300)
> quantmod::chart_Series(xtsv, name="VTI EMA Volatility")
```


Estimating *EMA* Variance Using Package *roll*

If the *time series* has non-zero *expected* mean, then the trailing *EMA* variance is a vector given by the estimator:

$$\sigma_t^2 = \frac{1}{k-1} \sum_{j=0}^{k-1} w_j (r_{t-j} - \bar{r}_t)^2$$

$$\bar{r}_t = \frac{1}{k} \sum_{j=0}^{k-1} w_j r_{t-j}$$

Where w_j is the vector of exponentially decaying weights:

$$w_j = \frac{\lambda^j}{\sum_{j=0}^{k-1} \lambda^j}$$

The function `roll_var()` from package *roll* calculates the trailing *EMA* variance.

```
> # Calculate trailing VTI variance using package roll
> library(roll) ## Load roll
> varv <- roll::roll_var(retp, weights=rev(weightv), width=lookb)
> colnames(varv) <- "VTI.variance"
> class(varv)
> head(varv)
> sum(is.na(varv))
> varv[1:(lookb-1)] <- 0
```

Trailing Realized Volatility Estimator

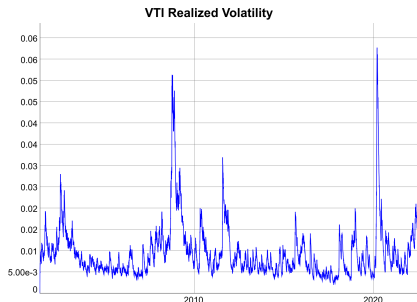
The function `HighFreq::run_var()` calculates the trailing mean and variance of the returns r_t , by recursively weighting the past variance estimates σ_{t-1}^2 , with the squared differences of the returns minus their trailing means $(r_t - \bar{r}_t)^2$, using the decay factor λ :

$$\bar{r}_t = \lambda \bar{r}_{t-1} + (1 - \lambda) r_t$$

$$\sigma_t^2 = \lambda^2 \sigma_{t-1}^2 + (1 - \lambda^2) (r_t - \bar{r}_t)^2$$

Where $\bar{r}_t$ and σ_t^2 are the trailing mean and variance at time t .

The decay factor λ determines how quickly the mean and variance estimates are updated, with smaller values of λ producing faster updating, giving more weight to recent prices, and vice versa.



```
> # Calculate realized variance recursively
> lambdaf <- 0.9
> volv <- HighFreq::run_var(retp, lambda=lambdaf)
> volv <- sqrt(volv[, 2])
> # Plot EMA volatility
> volv <- xts::xts(volv, order.by=datev)
> dygraphs::dygraph(volv, main="VTI Realized Volatility") %>%
+   dyOptions(colors="blue") %>% dyLegend(show="always", width=300)
```

Estimating Daily Volatility From Intraday Returns

The standard *close-to-close* volatility σ depends on the *Close* prices C_i from *OHLC* data:

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (r_i - \bar{r})^2$$

$$\bar{r} = \frac{1}{n} \sum_{i=0}^n r_i \quad r_i = \log\left(\frac{C_i}{C_{i-1}}\right)$$

But intraday time series of prices (for example `HighFreq::SPY` prices), can have large overnight jumps which inflate the volatility estimates.

So the overnight returns must be divided by the overnight time interval (in seconds), which produces per second returns.

The per second returns can be multiplied by 60 to scale them back up to per minute returns.

The function `zoo::index()` extracts the time index of a time series.

The function `xts::.index()` extracts the time index expressed in the number of seconds.

```
> library(HighFreq) ## Load HighFreq
> # Minutely SPY returns (unit per minute) single day
> # Minutely SPY volatility (unit per minute)
> retspy <- rutils::diffit(log(SPY["2012-02-13", 4]))
> sd(retspy)
> # SPY returns multiple days (includes overnight jumps)
> retspy <- rutils::diffit(log(SPY[, 4]))
> sd(retspy)
> # Table of time intervals - 60 second is most frequent
> indeks <- rutils::diffit(xts::.index(SPY))
> table(indeks)
> # SPY returns divided by the overnight time intervals (unit per s
> retspy <- retspy/indeks
> retspy[1] <- 0
> # Minutely SPY volatility scaled to unit per minute
> 60*sd(retspy)
```

Range Volatility Estimators of OHLC Time Series

Range estimators of return volatility utilize the high and low prices, and therefore have lower standard errors than the standard *close-to-close* estimator.

The *Garman-Klass* estimator uses the *low-to-high* price range, but it underestimates volatility because it doesn't account for *close-to-open* price jumps:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (0.5 \log(\frac{H_i}{L_i})^2 - (2 \log 2 - 1) \log(\frac{C_i}{O_i})^2)$$

The *Yang-Zhang* estimator accounts for *close-to-open* price jumps and has the lowest standard error among unbiased estimators:

$$\begin{aligned} \sigma^2 = & \frac{1}{n-1} \sum_{i=1}^n (\log(\frac{O_i}{C_{i-1}}) - \bar{r}_{co})^2 + \\ & 0.134 (\log(\frac{C_i}{O_i}) - \bar{r}_{oc})^2 + \\ & \frac{0.866}{n} \sum_{i=1}^n (\log(\frac{H_i}{O_i}) \log(\frac{H_i}{C_i}) + \log(\frac{L_i}{O_i}) \log(\frac{L_i}{C_i})) \end{aligned}$$

The *Yang-Zhang* (YZ) and *Garman-Klass-Yang-Zhang* (GKYZ) estimators are unbiased and have up to seven times smaller standard errors than the standard *close-to-close* estimator.

But in practice, prices are not observed continuously, so the price range is underestimated, and so is the variance when using the YZ and GKYZ range estimators.

Therefore in practice the YZ and GKYZ range estimators underestimate the volatility, and their standard errors are reduced less than by the theoretical amount, for the same reason.

The *Garman-Klass-Yang-Zhang* estimator is another very efficient and unbiased estimator, and also accounts for *close-to-open* price jumps:

$$\begin{aligned} \sigma^2 = & \frac{1}{n} \sum_{i=1}^n ((\log(\frac{O_i}{C_{i-1}}) - \bar{r})^2 + \\ & 0.5 \log(\frac{H_i}{L_i})^2 - (2 \log 2 - 1) (\log(\frac{C_i}{O_i}))^2) \end{aligned}$$

Calculating the Trailing Range Variance Using *HighFreq*

The function `HighFreq::calc_var_ohlcv()` calculates the *variance* of returns using several different range volatility estimators.

If the logarithms of the *OHLC* prices are passed into `HighFreq::calc_var_ohlcv()` then it calculates the variance of percentage returns, and if simple *OHLC* prices are passed then it calculates the variance of dollar returns.

The function `HighFreq::roll_var_ohlcv()` calculates the *trailing* variance of returns using several different range volatility estimators.

The functions `HighFreq::calc_var_ohlcv()` and `HighFreq::roll_var_ohlcv()` are very fast because they are written in C++ code.

The function `TTR::volatility()` calculates the range volatility, but it's significantly slower than `HighFreq::calc_var_ohlcv()`.

```
> library(HighFreq) ## Load HighFreq
> spy <- HighFreq::SPY["2008/2009"]
> # Calculate daily SPY volatility using package HighFreq
> sqrt(6.5*60*HighFreq::calcvar_ohlcv(log(spy),
+   method="yang_zhang"))
> # Calculate daily SPY volatility from minutely prices using package
> sqrt((6.5*60)*mean(na.omit(
+   TTR::volatility(spy, N=1, calc="yang.zhang"))^2))
> # Calculate trailing SPY variance using package HighFreq
> varv <- HighFreq::roll_var_ohlcv(log(spy), method="yang_zhang",
+   lookb=lookb)
> # Plot range volatility
> varv <- xts::xts(sqrt(varv), order.by=zoo::index(spy))
> dygraphs::dygraph(varv["2009-02"], main="SPY Trailing Range Volat
+   dyOptions(colors="blue") %>% dyLegend(show="always", width=300)
> # Benchmark the speed of HighFreq vs TTR
> library(microbenchmark)
> summary(microbenchmark(
+   ttr=TTR::volatility(rutils::etfenv$VTI, N=1, calc="yang.zhang"),
+   highfreq=HighFreq::calcvar_ohlcv(log(rutils::etfenv$VTI), method=
+   times=2)), c(1, 4, 5])
```

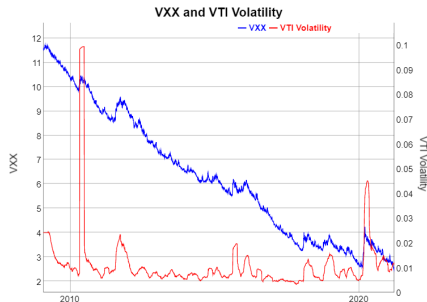
VXX Prices and the Trailing Volatility

The VXX ETF invests in *VIX* futures, so its price is tied to the level of the *VIX* index, with higher VXX prices corresponding to higher levels of the *VIX* index.

The trailing volatility of past returns moves in sympathy with the implied volatility and VXX prices, but with a lag.

But VXX prices exhibit a very strong downward trend which makes them hard to compare with the trailing volatility.

```
> # Calculate VXX log prices
> vxx <- na.omit(rutils::etfenv$prices$VXX)
> datev <- zoo::index(vxx)
> lookb <- 41
> vxx <- log(vxx)
> # Calculate trailing VTI volatility
> closep <- get("VTI", rutils::etfenv)[datev]
> closep <- log(closep)
> volv <- sqrt(HighFreq::roll_var_ohlcv(ohlcv=closep, lookb=lookb, s=
> volv[1:lookb] <- volv[lookb+1])
```

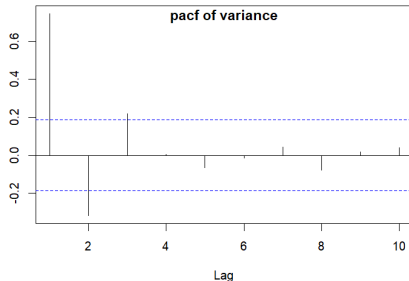
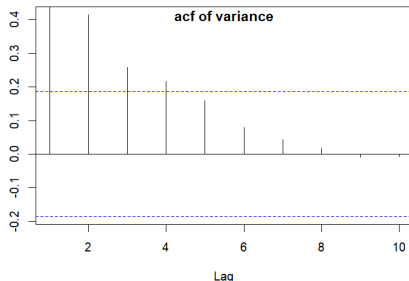


```
> # Plot dygraph of VXX and VTI volatility
> datav <- cbind(vxx, volv)
> colnames(datav)[2] <- "VTI Volatility"
> colv <- colnames(datav)
> caption <- "VXX and VTI Volatility"
> dygraphs::dygraph(datav[, 1:2], main=caption) %>%
+   dyAxis("y", label=colv[1], independentTicks=TRUE) %>%
+   dyAxis("y2", label=colv[2], independentTicks=TRUE) %>%
+   dySeries(name=colv[1], axis="y", strokeWidth=1, col="blue") %>%
+   dySeries(name=colv[2], axis="y2", strokeWidth=1, col="red") %>%
+   dyLegend(show="always", width=300)
```

Autocorrelation of Volatility

Variance calculated over non-overlapping intervals has very statistically significant autocorrelations.

```
> # Calculate VTI percentage returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Calculate VTI rolling variance
> lookb <- 21
> varv <- HighFreq::roll_var(retp, lookb=lookb)
> colnames(varv) <- "Variance"
> # Number of lookbv that fit over returns
> nrows <- NROW(retp)
> nagg <- nrows %/% lookb
> # Define end points with beginning stub
> endd <- c(0, nrows-lookb*nagg + (0:nagg)*lookb)
> nrows <- NROW(endd)
> # Subset variance to end points
> varv <- varv[endd]
> # Plot autocorrelation function
> rutils::plot_acf(varv, lag=10, main="ACF of Variance")
> # Plot partial autocorrelation
> pacf(varv, lag=10, main="PACF of Variance", ylab=NA)
```



The GARCH Volatility Model

The $GARCH(1,1)$ volatility model is defined by two coupled equations:

$$r_t = \sigma_{t-1} \xi_t$$

$$\sigma_t^2 = \omega + \beta \sigma_{t-1}^2 + \alpha r_t^2$$

The time-dependent variance σ_t^2 , is equal to the weighted average of the *realized* variance r_t^2 and the past variance σ_{t-1}^2 .

The source of uncertainty are the returns r_t , which are proportional to the standard normal innovations ξ_t .

The parameter α is the weight associated with recent realized variance updates, and β is the weight associated with the past variance.

The long-term expected value of the variance is proportional to the parameter ω :

$$\sigma^2 = \frac{\omega}{1 - \alpha - \beta}$$

So the sum of α plus β should be less than 1, otherwise the volatility is explosive.

NYU Professor [Robert Engle](#) was awarded the Nobel Prize in Economics for developing the *GARCH* volatility model.

```
> # Define GARCH parameters
> alphac <- 0.3; betac <- 0.5;
> omega <- 1e-4*(1 - alphac - betac)
> nrows <- 1000
> # Calculate matrix of standard normal innovations
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection") ##
> innov <- rnorm(nrows)
> retp <- numeric(nrows)
> varv <- numeric(nrows)
> varv[1] <- omega/(1 - alphac - betac)
> retp[1] <- sqrt(varv[1])*innov[1]
> # Simulate GARCH model
> for (i in 2:nrows) {
+   retp[i] <- sqrt(varv[i-1])*innov[i]
+   varv[i] <- omega + alphac*retp[i]^2 + betac*varv[i-1]
+ } ## end for
> # Simulate the GARCH process using Rcpp
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=matrix(innov))
> all.equal(garchsim, cbind(retp, varv), check.attributes=FALSE)
```

The *GARCH* process is path dependent, so it must be simulated using an explicit loop, and it's better to perform it in C++ instead of R.

GARCH Volatility Time Series

The simulated *GARCH* volatility exhibits spikes of volatility followed by an exponential decay.

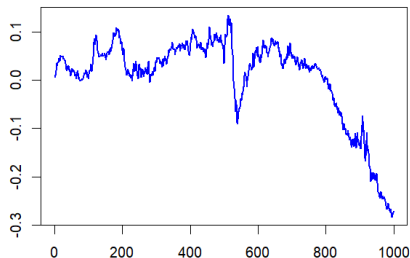
Larger values of α produce a stronger feedback between the simulated returns and variance, which produce larger variance spikes, which produce larger kurtosis.

The parameter α is the weight of the squared realized returns in the variance.

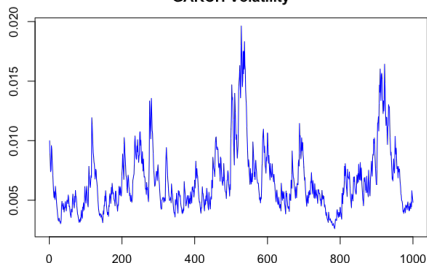
But the decay of the volatility in the *GARCH* model is faster than what is observed in practice.

```
> # Open plot window on Mac
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> # Set plot parameters to reduce whitespace around plot
> par(mar=c(2, 2, 3, 1), oma=c(0, 0, 0, 0))
> # Plot GARCH cumulative returns
> plot(cumsum(retp), t="l", col="blue", xlab="", ylab="",
+      main="GARCH Cumulative Returns")
> quartz.save("figure/garch_returns.png", type="png",
+             width=6, height=5)
> # Plot GARCH volatility
> plot(sqrt(varv), t="l", col="blue", xlab="", ylab="",
+      main="GARCH Volatility")
> quartz.save("figure/garch_volat.png", type="png",
+             width=6, height=5)
```

GARCH cumulative returns



GARCH Volatility



GARCH Returns Distribution

The return process r_t follows a normal distribution, *conditional* on the variance in the previous period σ_{t-1}^2 .

$$r_t = \sigma_{t-1} \xi_t$$

$$\sigma_t^2 = \omega + \beta \sigma_{t-1}^2 + \alpha r_t^2$$

But the *unconditional* distribution of returns is *not* normal, since their standard deviation is time-dependent, so they are *leptokurtic* (fat tailed).

The GARCH volatility model produces *leptokurtic* return distribution, with fat tails.

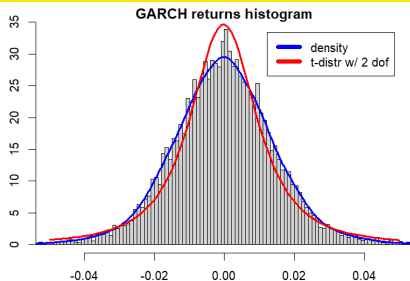
Student's *t-distribution* has fat tails, so it fits asset returns much better than the normal distribution.

Student's *t-distribution* with 3 degrees of freedom is often used to represent asset returns.

The function `fitdistr()` from package *MASS* fits a univariate distribution into a sample of data, by performing *maximum likelihood* optimization.

The function `hist()` calculates and plots a histogram, and returns its data *invisibly*.

```
> # Calculate kurtosis of GARCH returns
> mean(((retp-mean(retp))/sd(retp))^4)
> # Perform Jarque-Bera test of normality
> tseries::jarque.bera.test(retp)
```



```
> # Fit t-distribution into GARCH returns
> fitobj <- MASS::fitdistr(retp, densfun="t", df=2)
> locv <- fitobj$estimate[1]
> scalev <- fitobj$estimate[2]
> # Plot histogram of GARCH returns
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=200, xlim=c(-0.03, 0.03),
+   ylab="frequency", freq=FALSE, main="GARCH Returns Histogram")
> lines(density(retp, adjust=1.5), lwd=2, col="blue")
> curve(expr=dt((x-locv)/scalev, df=2)/scalev,
+   type="l", xlab="", ylab="", lwd=2,
+   col="red", add=TRUE)
> legend("topright", inset=-0, bty="n", y.intersp=0.4,
+   leg=c("density", "t-distr w/ 2 dof"),
+   lwd=6, lty=1, col=c("blue", "red"))
> quartz.save("figure/garch_hist.png", type="png", width=6, height=6)
```

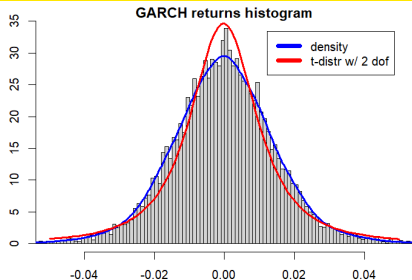
GARCH Model Simulation

The package `fGarch` contains functions for applying *GARCH* models.

The function `fGarch::garchSpec()` specifies a *GARCH* model.

The function `fGarch::garchSim()` simulates a *GARCH* model, but it uses its own random innovations, so its output is not reproducible.

```
> # Specify GARCH model
> garch_spec <- fGarch::garchSpec(model=list(ar=c(0, 0), omega=omeg
+   alpha=alphac, beta=betac))
> # Simulate GARCH model
> garch_sim <- fGarch::garchSim(spec=garch_spec, n=nrows)
> retp <- as.numeric(garch_sim)
> # Calculate kurtosis of GARCH returns
> moments::moment(retp, order=4) /
+   moments::moment(retp, order=2)^2
> # Perform Jarque-Bera test of normality
> tseries::jarque.bera.test(retp)
> # Plot histogram of GARCH returns
> histp <- hist(retp, col="lightgrey",
+   xlab="returns", breaks=200, xlim=c(-0.05, 0.05),
+   ylab="frequency", freq=FALSE,
+   main="GARCH Returns Histogram")
> lines(density(retp, adjust=1.5), lwd=3, col="blue")
```



```
> # Fit t-distribution into GARCH returns
> fitobj <- MASS::fitdistr(retp, densfun="t", df=2, lower=c(-1, 1e-7))
> locv <- fitobj$estimate[1]
> scalev <- fitobj$estimate[2]
> curve(expr=dt((x-locv)/scalev, df=2)/scalev,
+   type="l", xlab="", ylab="", lwd=3,
+   col="red", add=TRUE)
> legend("topright", inset=0.05, bty="n", y.intersp=0.4,
+   leg=c("density", "t-distr w/ 2 dof"),
+   lwd=6, lty=1, col=c("blue", "red"))
```

GARCH Returns Kurtosis

The expected value of the variance σ^2 of GARCH returns is proportional to the parameter ω :

$$\sigma^2 = \frac{\omega}{1 - \alpha - \beta}$$

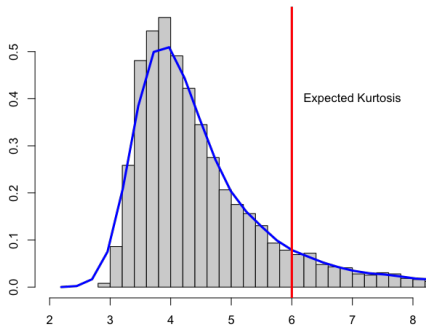
The expected value of the kurtosis κ of GARCH returns is equal to:

$$\kappa = 3 + \frac{6\alpha^2}{1 - 2\alpha^2 - (\alpha + \beta)^2}$$

The excess kurtosis $\kappa - 3$ is proportional to α^2 because larger values of the parameter α produce larger variance spikes which produce larger kurtosis.

The distribution of kurtosis is highly positively skewed, especially for short returns samples, so most kurtosis values will be significantly below their expected value.

Distribution of GARCH Kurtosis



```
> # Calculate variance of GARCH returns
> var(retp)
> # Calculate expected value of variance
> omega/(1 - alphac - betac)
> # Calculate kurtosis of GARCH returns
> mean(((retp-mean(retp))/sd(retp))^4)
> # Calculate expected value of kurtosis
> 3 + 6*alpha^2/(1-2*alpha^2-(alphac+betac)^2)
```

```
> # Calculate the distribution of GARCH kurtosis
> kurt <- sapply(1:1e4, function(x) {
+   garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+     beta=betac, innov=matrix(rnorm(nrows)))
+   retp <- garchsim[, 1]
+   c(var(retp), mean(((retp-mean(retp))/sd(retp))^4))
+ }) ## end sapply
> kurt <- t(kurt)
> apply(kurt, 2, mean)
> # Plot the distribution of GARCH kurtosis
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> par(mar=c(2, 2, 3, 1), oma=c(0, 0, 0, 0))
> histp <- hist(kurt[, 2], breaks=500, col="lightgrey",
+   xlim=c(2, 8), xlab="returns", ylab="frequency", freq=FALSE,
+   main="Distribution of GARCH Kurtosis")
```

GARCH Variance Estimation

The *GARCH* model can be used to estimate the trailing variance of empirical (historical) returns.

If the time series of returns r_t is given, then it can be used in the *GARCH(1,1)* formula to estimate the trailing variance σ_t^2 :

$$\sigma_t^2 = \omega + \beta\sigma_{t-1}^2 + \alpha r_t^2$$

If the simulated returns from the *GARCH(1,1)* model are used in the above formula, then it produces the simulated *GARCH(1,1)* variance.

But to estimate the trailing variance of historical returns, the parameters ω , α , and β must be first estimated through model calibration.

```
> # Simulate the GARCH process using Rcpp
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=matrix(innov))
> # Extract the returns
> retp <- garchsim[, 1]
> # Estimate the trailing variance from the returns
> varv <- numeric(nrows)
> varv[1] <- omega/(1 - alphac - betac)
> for (i in 2:nrows) {
+   varv[i] <- omega + alphac*retp[i]^2 +
+     betac*varv[i-1]
+ } ## end for
> all.equal(garchsim[, 2], varv, check.attributes=FALSE)
```

GARCH Model Calibration

GARCH models can be calibrated from the returns using the *maximum-likelihood* method.

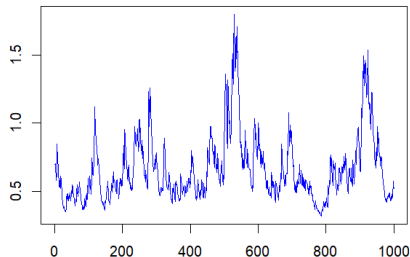
But it's a complex optimization procedure which requires a large amount of data for accurate results.

The function `fGarch::garchFit()` calibrates a *GARCH* model on a time series of returns.

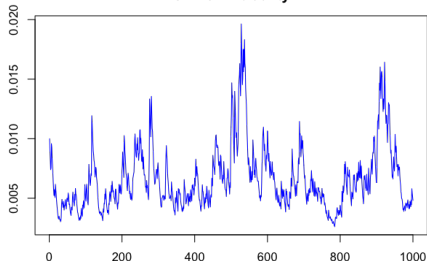
The function `garchFit()` returns an S4 object of class *fGARCH*, with multiple slots containing the *GARCH* model outputs and diagnostic information.

```
> library(fGarch)
> # Fit returns into GARCH
> garchfit <- fGarch::garchFit(data=retp)
> # Fitted GARCH parameters
> garchfit@fit$coef
> # Actual GARCH parameters
> c(mu=mean(retp), omega=omega, alpha=alphac, beta=betac)
> # Plot GARCH fitted volatility
> plot(sqrt(garchfit@fit$series$h), t="l",
+   col="blue", xlab="", ylab="",
+   main="GARCH Fitted Volatility")
> quartz.save("figure/garch_fGarch_fitted.png",
+   type="png", width=6, height=5)
```

GARCH fitted standard deviation



GARCH Volatility



GARCH Likelihood Function

Under the $GARCH(1,1)$ volatility model, the returns follow the process: $r_t = \sigma_{t-1}\xi_t$. (We can assume that the returns have been centered.)

So the *conditional* distribution of returns is normal with standard deviation equal to σ_{t-1} :

$$\phi(r_t, \sigma_{t-1}) = \frac{e^{-r_t^2/2\sigma_{t-1}^2}}{\sqrt{2\pi}\sigma_{t-1}}$$

The *log-likelihood* function $\mathcal{L}(\omega, \alpha, \beta | r_t)$ for the normally distributed returns is therefore equal to:

$$\mathcal{L}(\omega, \alpha, \beta | r_t) = - \sum_{t=1}^n \left(\frac{r_t^2}{\sigma_{t-1}^2} + \log(\sigma_{t-1}^2) \right)$$

The *log-likelihood* depends on the $GARCH(1,1)$ parameters ω , α , and β because the trailing variance σ_t^2 depends on the $GARCH(1,1)$ parameters:

$$\sigma_t^2 = \omega + \beta\sigma_{t-1}^2 + \alpha r_t^2$$

The $GARCH$ process must be simulated using an explicit loop, so it's better to perform it in C++ instead of R.

```
> # Define likelihood function
> likefun <- function(omega, alphac, betac) {
+   ## Estimate the trailing variance from the returns
+   varv <- numeric(nrows)
+   varv[1] <- omega/(1 - alphac - betac)
+   for (i in 2:nrows) {
+     varv[i] <- omega + alphac*ret[p[i]]^2 + betac*varv[i-1]
+   } ## end for
+   varv <- ifelse(varv > 0, varv, 0.000001)
+   ## Lag the variance
+   varv <- rutils::lagit(varv, pad_zeros=FALSE)
+   ## Calculate the likelihood
+   -sum(ret[p]^2/varv + log(varv))
+ } ## end likefun
> # Calculate the likelihood in R
> likefun(omega, alphac, betac)
> # Calculate the likelihood in Rcpp
> HighFreq::lik_garch(omega=omega, alpha=alphac,
+   beta=betac, returns=matrix(ret[p]))
> # Benchmark speed of likelihood calculations
> library(microbenchmark)
> summary(microbenchmark(
+   Rcode=likefun(omega, alphac, betac),
+   Rcpp=HighFreq::lik_garch(omega=omega, alpha=alphac, beta=betac,
+   ), times=10)[, c(1, 4, 5)])
```

GARCH Likelihood Function Matrix

The $GARCH(1,1)$ *log-likelihood* function depends on three parameters $\mathcal{L}(\omega, \alpha, \beta | r_t)$.

The more parameters the harder it is to find their optimal values using optimization.

We can simplify the optimization task by assuming that the expected variance is equal to the realized variance:

$$\sigma^2 = \frac{\omega}{1 - \alpha - \beta} = \frac{1}{n-1} \sum_{t=1}^n (r_t - \bar{r})^2$$

This way the *log-likelihood* becomes a function of only two parameters, say α and β .

```
> # Calculate the variance of returns
> retp <- garchsim[, 1, drop=FALSE]
> varv <- var(retp)
> retp <- (retp - mean(retp))
> # Calculate likelihood as function of alpha and betac parameters
> likefun <- function(alphac, betac) {
+   omega <- variance*(1 - alpha - betac)
+   -HighFreq::lik_garch(omega=omega, alpha=alphac, beta=betac, retp=retp)
+ } ## end likefun
> # Calculate matrix of likelihood values
> alphas <- seq(from=0.15, to=0.35, len=50)
> betac <- seq(from=0.35, to=0.5, len=50)
> likmat <- sapply(alphas, function(alphac) sapply(betac,
+   function(betac) likefun(alphac, betac)))
```

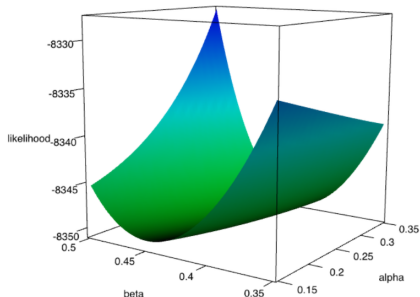

GARCH Likelihood Perspective Plot

The perspective plot shows that the *log-likelihood* is much more sensitive to the β parameter than to α .

The function `rgl::persp3d()` plots an *interactive* 3d surface plot of a *vectorized* function or a matrix.

The optimal values of α and β can be found approximately using a grid search on the *log-likelihood* matrix.

```
> # Set rgl options and load package rgl
> options(rgl.useNULL=TRUE); library(rgl)
> # Draw and render 3d surface plot of likelihood function
> ncols <- 100
> color <- rainbow(ncols, start=2/6, end=4/6)
> zcols <- cut(likmat, ncols)
> rgl::persp3d(alphaacs, betac, likmat, col=color[zcols],
+   xlab="alpha", ylab="beta", zlab="likelihood")
> rgl::rglwidget(elementId="plot3drgl", width=700, height=700)
> # Perform grid search
> coord <- which(likmat == min(likmat), arr.ind=TRUE)
> c(alphaacs[coord[2]], betac[coord[1]])
> likmat[coord]
> likefun(alphaacs[coord[2]], betac[coord[1]])
> # Optimal and actual parameters
> options(scipen=2) ## Use fixed not scientific notation
> cbind(actual=c(alpha=alphac, beta=betac, omega=omega),
+   optimal=c(alphaacs[coord[2]], betac[coord[1]], variance*(1 - sum(alphaacs[coord[2]], betac[coord[1]]))))
```



GARCH Likelihood Function Optimization

The flat shape of the *GARCH* likelihood function makes it difficult for steepest descent optimizers to find the best parameters.

The function `DEoptim()` from package *DEoptim* performs *global* optimization using the *Differential Evolution* algorithm.

Differential Evolution is a genetic algorithm which evolves a population of solutions over several generations:

<https://link.springer.com/content/pdf/10.1023/A:1008202821328.pdf>

The first generation of solutions is selected randomly.

Each new generation is obtained by combining the best solutions from the previous generation.

The *Differential Evolution* algorithm is well suited for very large multi-dimensional optimization problems, such as portfolio optimization.

Gradient optimization methods are more efficient than *Differential Evolution* for smooth objective functions with no local minima.

```
> # Define vectorized likelihood function
> likefun <- function(x, retp) {
+   alphac <- x[1]; betac <- x[2]; omega <- x[3]
+   -HighFreq::lik_garch(omega=omega, alpha=alphac, beta=betac, retp=retp)
+ } ## end likefun
> # Initial parameters
> initp <- c(alphac=0.2, beta=0.4, omega=varv/0.2)
> # Find max likelihood parameters using steepest descent optimizer
> fitobj <- optim(par=initp,
+   fn=likefun, ## Log-likelihood function
+   method="L-BFGS-B", ## Quasi-Newton method
+   returns=retp,
+   upper=c(0.35, 0.55, varv), ## Upper constraint
+   lower=c(0.15, 0.35, varv/100)) ## Lower constraint
> # Optimal and actual parameters
> cbind(actual=c(alphac=alphac, beta=betac, omega=omega),
+   optimal=c(fitobj$par["alpha"], fitobj$par["beta"], fitobj$par["omega"]))
> # Find max likelihood parameters using DEoptim
> optim1 <- DEoptim(fn=likefun,
+   upper=c(0.35, 0.55, varv), ## Upper constraint
+   lower=c(0.15, 0.35, varv/100), ## Lower constraint
+   returns=retp,
+   control=list(trace=FALSE, itermax=1000, parallelType=1))
> # Optimal and actual parameters
> cbind(actual=c(alphac=alphac, beta=betac, omega=omega),
+   optimal=c(optim1$optim$bestmem[1], optim1$optim$bestmem[2], optim1$optim$bestmem[3]))
```

GARCH Variance of Stock Returns

The *GARCH* model can be used to estimate the trailing variance of empirical (historical) returns.

If the time series of returns r_t is given, then it can be used in the *GARCH(1,1)* formula to estimate the trailing variance σ_t^2 :

$$\sigma_t^2 = \omega + \beta\sigma_{t-1}^2 + \alpha r_t^2$$

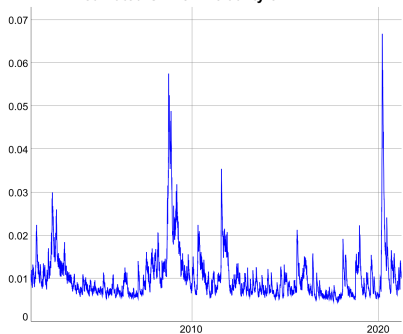
The *GARCH* estimator of the trailing variance is a generalization of the *Exponentially Weighted Moving Average (EMA)* variance estimator:

$$\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1 - \lambda)r_t^2$$

The main difference is that the *GARCH* model has an equilibrium value of variance σ^2 .

```
> # Calculate VTI returns
> retp <- na.omit(rutils::etfenv$returns$VTI)
> # Find max likelihood parameters using DEoptim
> optim1 <- DEoptim::DEoptim(fn=likefun,
+   upper=c(0.4, 0.9, varv), ## Upper constraint
+   lower=c(0.1, 0.5, varv/100), ## Lower constraint
+   returns=retp,
+   control=list(trace=FALSE, itermax=1000, parallelType=1))
> # Optimal parameters
> paramv <- unname(optim1$optim$bestmem)
> alphac <- paramv[1]; betac <- paramv[2]; omega <- paramv[3]
> c(alphac, betac, omega)
> # Equilibrium GARCH variance
> omega/(1 - alphac - betac)
> drop(var(retp))
```

Estimated GARCH Volatility of VTI



```
> # Estimate the GARCH volatility of VTI returns
> nrow <- NROW(retp)
> varv <- numeric(nrow)
> varv[1] <- omega/(1 - alphac - betac)
> for (i in 2:nrow) {
+   varv[i] <- omega + alphac*retp[i]^2 + betac*varv[i-1]
+ } ## end for
> # Estimate the GARCH volatility using Rcpp
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=retp, is_random=FALSE)
> all.equal(garchsim[, 2], varv, check.attributes=FALSE)
> # Plot dygraph of the estimated GARCH volatility
> dygraphs::dygraph(xts::xts(sqrt(varv), zoo::index(retp)),
+   main="Estimated GARCH Volatility of VTI") %>%
+   dyOptions(colors="blue") %>% dvLegend(show="always". width=300)
```

GARCH Variance Forecasts

The *GARCH* model can't forecast the volatility spikes. It only forecasts the exponential decay of the volatility after a spike.

The one-step-ahead forecast of the squared returns is equal to their expected value: $r_{t+1}^2 = \mathbb{E}[(\sigma_t \xi_t)^2] = \sigma_t^2$.

The variance forecasts depend on the previous variance: $\sigma_{t+1}^2 = \mathbb{E}[\omega + \alpha r_{t+1}^2 + \beta \sigma_t^2] = \omega + (\alpha + \beta) \sigma_t^2$

The variance forecasts gradually settle to the equilibrium value σ^2 , such that the forecast is equal to itself: $\sigma^2 = \omega + (\alpha + \beta) \sigma^2$.

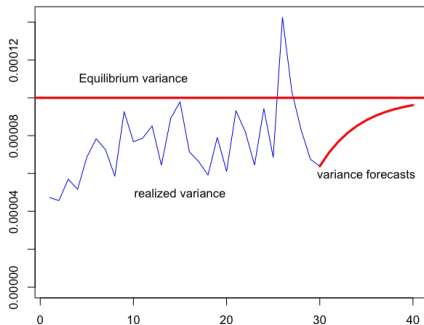
This gives: $\sigma^2 = \frac{\omega}{1 - \alpha - \beta}$, which is the long-term expected value of the variance.

So the variance forecasts decay exponentially to their equilibrium value σ^2 at the decay rate equal to $(\alpha + \beta)$:

$$\sigma_{t+1}^2 - \sigma^2 = (\alpha + \beta)(\sigma_t^2 - \sigma^2)$$

```
> # Simulate GARCH model
> garchsim <- HighFreq::sim_garch(omega=omega, alpha=alphac,
+   beta=betac, innov=matrix(innov))
> varv <- garchsim[, 2]
> # Calculate the equilibrium variance
> vareq <- omega/(1 - alphac - betac)
> # Calculate the variance forecasts
> varf <- numeric(10)
> varf[1] <- vareq + (alphac + betac)*(xts::last(varv) - vareq)
> for (i in 2:10) {
+   varf[i] <- vareq + (alphac + betac)*(varf[i-1] - vareq)
+ } ## end for
```

GARCH Variance Forecasts



```
> # Open plot window on Mac
> dev.new(width=6, height=5, noRStudioGD=TRUE)
> par(mar=c(2, 2, 3, 1), oma=c(0, 0, 0, 0))
> # Plot GARCH variance forecasts
> plot(tail(varv, 30), t="l", col="blue", xlab="", ylab="",
+   xlim=c(1, 40), ylim=c(0, max(tail(varv, 30))),
+   main="GARCH Variance Forecasts")
> text(x=15, y=0.5*vareq, "realized variance")
> lines(x=30:40, y=c(xts::last(varv), varf), col="red", lwd=3)
> text(x=35, y=0.6*vareq, "variance forecasts")
> abline(h=vareq, lwd=3, col="red")
> text(x=10, y=1.1*vareq, "Equilibrium variance")
> quartz.save("figure/garch_forecast.png", type="png", width=6, height=5)
```

Brownian Motion

Brownian motion B_T is a stochastic process, with the increments dB_t which are independent and normally distributed, with mean zero and variance equal to dt .

$$dB_t = \xi_t \sqrt{dt}$$

Where the ξ_t are random and independent *innovations* following the standard normal distribution $\phi(0, 1)$, with the expected values: $\mathbb{E}[\xi_t] = 0$, $\mathbb{E}[\xi_t^2] = 1$, and $\mathbb{E}[\xi_{t1}\xi_{t2}] = 0$.

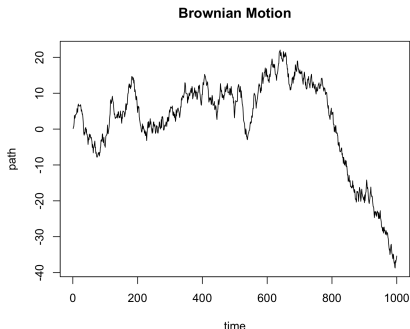
The Brownian motion path B_T is equal to the sum of its increments dB_t :

$$B_T = \sum_{i=1}^n dB_t$$

Where the number of time increments n is equal to the total time of evolution T divided by the increment size dt : $n = T/dt$.

The variance of Brownian motion is equal to the time of its evolution T :

$$\sigma^2 = \mathbb{E}[(\sum_{i=1}^n \xi_i \sqrt{dt})^2] = \sum_{i=1}^n \mathbb{E}[\xi_i^2] dt = T$$



```
> # Simulate a Brownian motion path
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> pathv <- cumsum(rnorm(nrows))
> plot(pathv, type="l", xlab="time", ylab="path",
+       main="Brownian Motion")
```

The Maximum Value of Brownian Motion

The distribution of the maximum value m of a Brownian motion path B_t can be calculated using the *reflection principle*.

The *reflection principle* states that the mirror image (reflection) of a Brownian motion path has the same probability as the original path.

The probability that the Brownian motion path B_t is at some point greater than some value m is the sum of the joint probability, that after the Brownian motion reaches the value m , it ends up greater than m , plus the joint probability that it ends up less than m :

$$p(B_t > m) = p((B_t > m) \& (B_T > m)) + p((B_t > m) \& (B_T < m))$$

By the *reflection principle*, both probabilities are equal, and also $p((B_t > m) \& (B_T > m)) = p(B_T > m)$, so that:

$$p(B_t > m) = 2p(B_T > m)$$

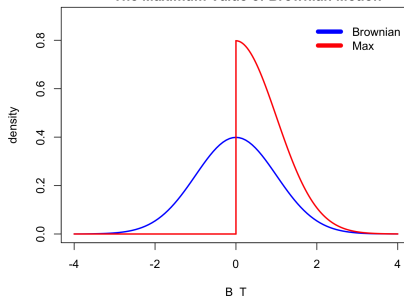
And the probability density of the maximum value m is equal to:

$$\varphi(m) = \sqrt{\frac{2}{\pi T}} e^{-\frac{m^2}{2T}}$$

The average value of the maximum value is equal to:

$$\bar{m} = \sqrt{\frac{2T}{\pi}}$$

Probability Density of
The Maximum Value of Brownian Motion



```
> # Plot the density of Brownian Motion
> curve(expr=dnorm(x), xlim=c(-4, 4), ylim=c(0, 0.9),
+       xlab="B_T", ylab="density", lwd=2, col="blue")
> # Plot the density of the maximum of Brownian Motion
> curve(expr=2*dnorm(x), xlim=c(0, 4), xlab="", ylab="",
+       lwd=2, col="red", add=TRUE)
> lines(x=c(0, 0), y=c(0, sqrt(2/pi)), lwd=2, col="red")
> lines(x=c(-4, 0), y=c(0, 0), lwd=2, col="red")
> title(main="Probability Density of
+ The Maximum Value of Brownian Motion", line=0.5)
> legend("topright", inset=0.0, bty="n", y.intersp=0.4,
+       title=NULL, c("Brownian", "Max"), lwd=6,
+       col=c("blue", "red"))
```

The Range of Brownian Motion

The range of a Brownian motion path B_t is equal to the difference between its maximum value minus its minimum value. The range is also called the drawdown.

The probability density of the range r is equal to the infinite series:

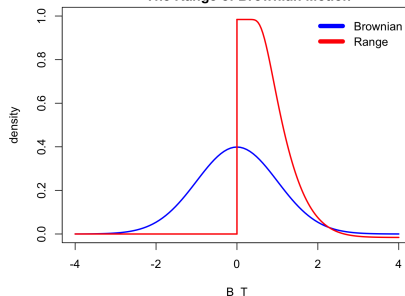
$$p(r) = 2 \sum_{n=1}^{\infty} \frac{\sin(n-0.5)\pi}{(n-0.5)\pi} \left(1 - e^{-\frac{(n-0.5)^2 \pi^2 T}{2r^2}}\right)$$

The average value of the range is equal to:

$$\bar{r} = \sqrt{\frac{\pi T}{2}}$$

```
> # Series element
> fun1 <- function(n, r) { 2*sin((n-0.5)*pi)/((n-0.5)*pi) *
+   (1-exp(-(n-0.5)^2*pi^2/2/r^2)) }
> # fun2 <- function(x) { sum(sapply(1:10, function(n) fun1(n, x))) }
> # fun2 <- function(x) { fun1(1, x) + fun1(2, x) + fun1(3, x) + fi
> # Sum of fun1
> fun2 <- function(x) {
+   valf <- 0
+   for (n in 1:20) { valf <- valf + fun1(n, x) }
+   return(valf)
+ } ## end fun2
> # Theoretical average value of the range
> fun2(2)
> # Average value of the range from integration (not quite close)
> integrate(fun2, lower=0.01, upper=4)
```

Probability Density of
The Range of Brownian Motion



```
> # Plot the density of Brownian Motion
> curve(expr=dnorm(x), xlim=c(-4, 4), ylim=c(0, 1.0),
+   xlab="B_T", ylab="density", lwd=2, col="blue")
> # Plot the density of the range of Brownian Motion
> curve(expr=fun2(x), xlim=c(0, 4), xlab="", ylab="",
+   lwd=2, col="red", add=TRUE)
> lines(x=c(0, 0), y=c(0, fun2(0.01)), lwd=2, col="red")
> lines(x=c(-4, 0), y=c(0, 0), lwd=2, col="red")
> title(main="Probability Density of
+ The Range of Brownian Motion", line=0.5)
> legend("topright", inset=0.0, bty="n", y.intersp=0.7,
+   title=NULL, c("Brownian", "Range"), lwd=6,
+   col=c("blue", "red"))
```

Geometric Brownian Motion

If the percentage asset returns $r_t dt = d \log p_t$ follow *Brownian motion*:

$$r_t dt = d \log p_t = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dB_t$$

Then asset prices p_t follow *Geometric Brownian motion* (GBM):

$$dp_t = \mu p_t dt + \sigma p_t dB_t$$

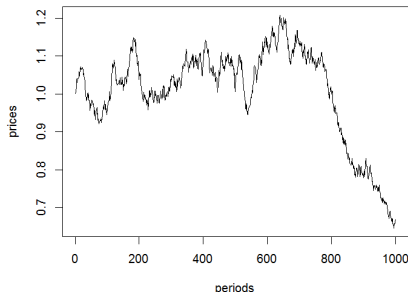
Where σ is the volatility of asset returns, and B_t is a *Brownian Motion*, with dB_t following the normal distribution $\phi(0, \sqrt{dt})$, with the volatility $\sqrt{dt}$, equal to the square root of the time increment dt .

The solution of *Geometric Brownian motion* is equal to:

$$p_t = p_0 \exp\left[\left(\mu - \frac{\sigma^2}{2}\right)t + \sigma B_t\right]$$

The convexity correction: $-\frac{\sigma^2}{2}$ ensures that the growth rate of prices is equal to μ , (according to Ito's lemma).

geometric Brownian motion



```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 1000
> # Simulate geometric Brownian motion
> retp <- sigmav*rnorm(nrows) + drift - sigmav^2/2
> pricev <- exp(cumsum(retp))
> plot(pricev, type="l", xlab="time", ylab="prices",
+      main="Geometric Brownian Motion")
```


Simulating Random OHLC Prices

Random OHLC prices are useful for testing financial models.

The function `sample()` selects a random sample from a vector of data elements.

The function `sample()` with `replace=TRUE` selects samples with replacement (the default is `replace=FALSE`).

```
> # Simulate geometric Brownian motion
> sigmav <- 0.01/sqrt(48)
> drift <- 0.0
> nrows <- 1e4
> datev <- seq(from=as.POSIXct(paste(Sys.Date()-250, "09:30:00")),
+   length.out=nrows, by="30 min")
> pricev <- exp(cumsum(sigmav*rnorm(nrows) + drift - sigmav^2/2))
> pricev <- xts(pricev, order.by=datev)
> pricev <- cbind(pricev,
+   volume=sample(x=10*(2:18), size=nrows, replace=TRUE))
> # Aggregate to daily OHLC data
> ohlc <- xts::to.daily(pricev)
> quantmod::chart_Series(ohlc, name="random prices")
> # dygraphs candlestick plot using pipes syntax
> library(dygraphs)
> dygraphs::dygraph(ohlc[, 1:4]) %>% dyCandlestick()
> # dygraphs candlestick plot without using pipes syntax
> dygraphs::dyCandlestick(dygraphs::dygraph(ohlc[, 1:4]))
```



The Log-normal Probability Distribution

If x follows the *Normal* distribution $\phi(x, \mu, \sigma)$, then the exponential of x : $y = e^x$ follows the *Log-normal* distribution $\log \phi()$:

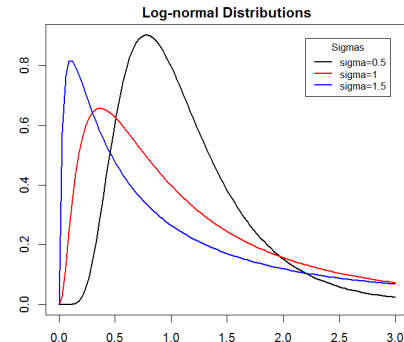
$$\log \phi(y, \mu, \sigma) = \frac{\exp(-(\log y - \mu)^2 / 2\sigma^2)}{y\sigma\sqrt{2\pi}}$$

With mean equal to: $\bar{y} = \mathbb{E}[y] = e^{(\mu + \sigma^2/2)}$, and median equal to: $\tilde{y} = e^\mu$

With variance equal to: $\sigma_y^2 = (e^{\sigma^2} - 1)e^{(2\mu + \sigma^2)}$, and skewness (third moment) equal to:

$$\varsigma = \mathbb{E}[(y - \mathbb{E}[y])^3] = (e^{\sigma^2} + 2)\sqrt{e^{\sigma^2} - 1}$$

```
> # Standard deviations of log-normal distribution
> sigmavs <- c(0.5, 1, 1.5)
> # Create plot colors
> colorv <- c("black", "red", "blue")
> # Plot all curves
> for (indeks in 1:NROW(sigmavs)) {
+   curve(expr=dlnorm(x, sdlog=sigmavs[indeks]),
+         type="l", lwd=2, xlim=c(0, 3),
+         xlab="", ylab="", col=colorv[indeks],
+         add=as.logical(indeks-1))
+ } ## end for
```



```
> # Add title and legend
> title(main="Log-normal Distributions", line=0.5)
> legend("topright", inset=0.05, title="Sigmas",
+       paste("sigma", sigmavs, sep=""),
+       cex=0.8, lwd=2, lty=rep(1, NROW(sigmavs)),
+       col=colorv)
```

The Standard Deviation of Log-normal Prices

If percentage asset returns are *normally* distributed and follow *Brownian motion*, then asset prices follow *Geometric Brownian motion*, and they are *Log-normally* distributed at every point in time.

The standard deviation of *log-normal* prices is equal to the return volatility σ_r times the square root of time:
 $\sigma = \sigma_r \sqrt{t}$.

The *Log-normal* distribution has a strong positive skewness (third moment) equal to:

$$\varsigma = \mathbb{E}[(y - \mathbb{E}[y])^3] = (e^{\sigma^2} + 2)\sqrt{e^{\sigma^2} - 1}$$

For large standard deviation, the skewness increases exponentially with the standard deviation and with time: $\varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$

```
> # Return volatility of VTI etf
> sigmav <- sd(rutils::diffit(log(rutils::etfenv$VTI[, 4])))
> sigma2 <- sigmav^2
> nrows <- NROW(rutils::etfenv$VTI)
> # Standard deviation of log-normal prices
> sqrt(nrows)*sigmav
```



```
> # Skewness of log-normal prices
> calcskew <- function(t) {
+   expv <- exp(t*sigma2)
+   (expv + 2)*sqrt(expv - 1)
+ } ## end calcskew
> curve(expr=calcskew, xlim=c(1, nrows), lwd=3,
+ xlab="Number of days", ylab="Skewness", col="blue",
+ main="Skewness of Log-normal Prices
+ as a Function of Time")
```

The Mean and Median of *Log-normal* Prices

The mean of the *Log-normal* distribution:
 $\bar{y} = E[y] = \exp(\mu + \sigma^2/2)$ is greater than its median,
 which is equal to: $\tilde{y} = \exp(\mu)$.

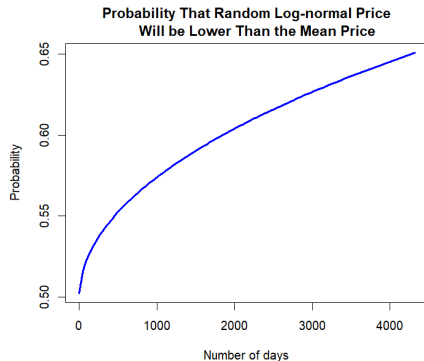
So if stock prices follow *Geometric Brownian motion*
 and are distributed *log-normally*, then a stock selected
 at random will have a high probability of having a lower
 price than the mean expected price.

The cumulative *Log-normal* probability distribution is
 equal to $F(x) = \Phi\left(\frac{\log y - \mu}{\sigma}\right)$, where $\Phi()$ is the
 cumulative standard normal distribution.

So the probability that the price of a randomly selected
 stock will be lower than the mean price is equal to
 $F(\bar{y}) = \Phi(\sigma/2)$.

Therefore an investor without skill, who selects stocks
 at random, has a high probability of underperforming
 the index.

Performing as well as the index requires *significant*
 investment skill, while outperforming the index requires
exceptional investment skill.



```
> # Probability that random log-normal price will be lower than the
> curve(expr=pnorm(sigmav*sqrt(x)/2),
+ xlim=c(1, nrow), lwd=3,
+ xlab="Number of days", ylab="Probability", col="blue",
+ main="Probability That Random Log-normal Price
+ Will be Lower Than the Mean Price")
```

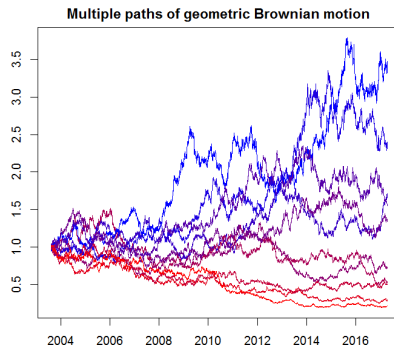
Paths of Geometric Brownian Motion

The standard deviation of *log-normal* prices σ is equal to the volatility of returns σ_r times the square root of time: $\sigma = \sigma_r \sqrt{t}$.

For large standard deviation, the skewness ς increases exponentially with the standard deviation and with

time: $\varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$

```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 5000
> npaths <- 10
> # Simulate multiple paths of geometric Brownian motion
> pricev <- rnorm(npaths*nrows, sd=sigmav) + drift - sigmav^2/2
> pricev <- matrix(pricev, nc=npaths)
> pricev <- exp(matrixStats::colCumsums(pricev))
> # Create xts time series
> pricev <- xts(pricev, order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()),
+               order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()),
+               order.by=seq.Date(Sys.Date()-nrows+1, Sys.Date()))
> # Sort the columns according to largest terminal values
> pricev <- pricev[, order(pricev[nrows, ])]
> # Plot xts time series
> colorv <- colorRampPalette(c("red", "blue"))(NCOL(pricev))
> par(mar=c(3, 3, 2, 2), oma=c(0, 0, 0, 0))
> plot.zoo(pricev, main="Multiple paths of geometric Brownian motion",
+          xlab=NA, ylab=NA, plot.type="single", col=colorv)
```



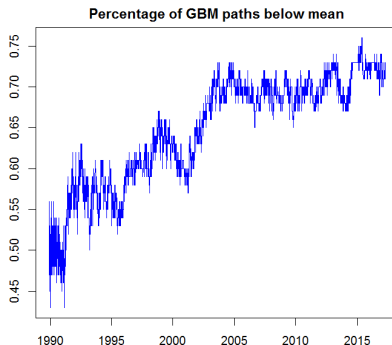
Distribution of Paths of Geometric Brownian Motion

Prices following *Geometric Brownian motion* have a large positive skewness, so that the expected value of prices is skewed by a few paths with very high prices, while the prices of the majority of paths are below their expected value.

For large standard deviation, the skewness ς increases exponentially with the standard deviation and with

$$\text{time: } \varsigma \propto e^{1.5\sigma^2} = e^{1.5t\sigma_r^2}$$

```
> # Define the daily volatility and growth rate
> sigmav <- 0.01; drift <- 0.0; nrows <- 10000
> npaths <- 100
> # Simulate multiple paths of geometric Brownian motion
> pricev <- rnorm(npaths*nrows, sd=sigmav) + drift - sigmav^2/2
> pricev <- matrix(pricev, nc=npaths)
> pricev <- exp(matrixStats::colCumsums(pricev))
> # Calculate fraction of paths below the expected value
> fractv <- rowSums(pricev < 1.0) / npaths
> # Create xts time series of percentage of paths below the expected value
> fractv <- xts(fractv, order.by=seq.Date(Sys.Date()-NROW(fractv)+1, Sys.Date(), by=1))
> # Plot xts time series of percentage of paths below the expected value
> par(mar=c(3, 3, 2, 2), oma=c(0, 0, 0, 0))
> plot.zoo(fractv, main="Percentage of GBM paths below mean",
+         xlab=NA, ylab=NA, col="blue")
```

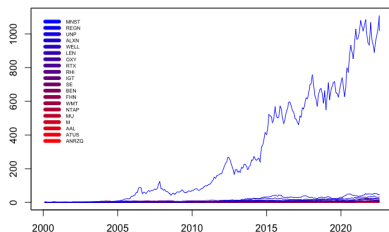


Time Evolution of Stock Prices

Stock prices evolve over time similar to *Geometric Brownian motion*, and they also exhibit a very skewed distribution of prices.

```
> # Load S&P500 stock prices
> load("/Users/jerzy/Develop/lecture_slides/data/sp500.RData")
> ls(sp500env)
> # Extract the closing prices
> pricev <- eapply(sp500env, quantmod::Cl)
> # Flatten the prices into a single xts series
> pricev <- rutils::do_call(cbind, pricev)
> # Carry forward and backward non-NA prices
> pricev <- zoo::na.locf(pricev, na.rm=FALSE)
> pricev <- zoo::na.locf(pricev, fromLast=TRUE)
> sum(is.na(pricev))
> # Drop ".Close" from column names
> colnames(pricev)
> colnames(pricev) <- rutils::get_name(colnames(pricev))
> # Or
> # colnames(pricev) <- do.call(rbind,
> #   strsplit(colnames(pricev), split="["))[, 1]
> # Select prices after the year 2000
> pricev <- pricev["2000/", ]
> # Scale the columns so that prices start at 1
> pricev <- lapply(pricev, function(x) x/as.numeric(x[1]))
> pricev <- rutils::do_call(cbind, pricev)
> # Sort the columns according to the final prices
> nrowv <- NROW(pricev)
> ordern <- order(pricev[nrowv, ])
> pricev <- pricev[, ordern]
> # Select 20 symbols
> symbolv <- colnames(pricev)
> symbolv <- symbolv[round(seq.int(from=1, to=NROW(symbolv), length.out=20))]
```

20 S&P500 Stock Prices (scaled)



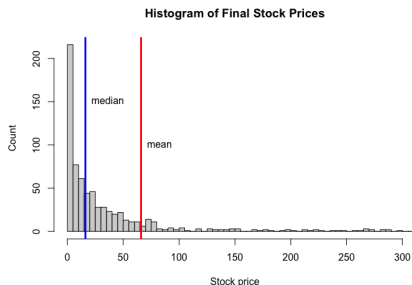
```
> # Plot xts time series of prices
> colorv <- colorRampPalette(c("red", "blue"))(NROW(symbolv))
> endd <- rutils::calc_endpoints(pricev, interval="weeks")
> plot.zoo(pricev[endd, symbolv], main="20 S&P500 Stock Prices (scaled)",
+   xlab=NA, ylab=NA, plot.type="single", col=colorv)
> legend(x="topleft", inset=0.02, cex=0.5, bty="n", y.intersp=0.5,
+   legend=rev(symbolv), col=rev(colorv), lwd=6, lty=1)
```

Distribution of Final Stock Prices

The distribution of the final stock prices is extremely skewed, with over 80% of the *S&P500* constituent stocks from 1990 now below the average price of that portfolio.

The *mean* of the final stock prices is much greater than the *median*.

```
> # Calculate the final stock prices
> pricef <- drop(zoo::coredata(pricev[nrows, ]))
> # Calculate the mean and median stock prices
> max(pricef); min(pricef)
> which.max(pricef)
> which.min(pricef)
> mean(pricef)
> median(pricef)
> # Calculate the percentage of stock prices below the mean
> sum(pricef < mean(pricef))/NROW(pricef)
```



```
> # Plot a histogram of final stock prices
> hist(pricef, breaks=1e3, xlim=c(0, 300),
+       xlab="Stock price", ylab="Count",
+       main="Histogram of Final Stock Prices")
> # Plot a histogram of final stock prices
> abline(v=median(pricef), lwd=3, col="blue")
> text(x=median(pricef), y=150, lab="median", pos=4)
> abline(v=mean(pricef), lwd=3, col="red")
> text(x=mean(pricef), y=100, lab="mean", pos=4)
```


Distribution of Stock Prices Over Time

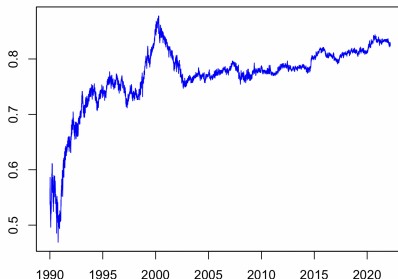
Usually, a small number of stocks in an index reach very high prices, while the prices of the majority of stocks remain below the index price (the average price of the index portfolio).

For example, the current prices of over 80% of the *S&P500* constituent stocks from 1990 are now below the average price of that portfolio.

Therefore an investor without skill, who selects stocks at random, has a high probability of underperforming the index, because they will most likely miss selecting the best performing stocks.

Performing as well as the index requires *significant* investment skill, while outperforming the index requires *exceptional* investment skill.

Percentage of S&P500 Stock Prices Below the Average Price



```
> # Calculate average of valid stock prices
> validp <- (pricev != 1) ## Valid stocks
> nstocks <- rowSums(validp)
> nstocks[1] <- NCOL(pricev)
> indeks <- rowSums(pricev*validp)/nstocks
> # Calculate fraction of stock prices below the average price
> fractv <- rowSums((pricev < indeks) & validp)/nstocks
> # Create xts time series of average stock prices
> indeks <- xts(indeks, order.by=zoo::index(pricev))
```

```
> dev.new(width=6, height=4, noRStudioGD=TRUE)
> # x11(width=6, height=4)
> # Plot xts time series of average stock prices
> plot.zoo(indeks, main="Average S&P500 Stock Prices (normalized fr
+   xlab=NA, ylab=NA, col="blue")
> # Create xts time series of percentage of stock prices below the a
> fractv <- xts(fractv, order.by=zoo::index(pricev))
> # Plot percentage of stock prices below the average price
> plot.zoo(fractv[-(1:2)],,
+   main="Percentage of S&P500 Stock Prices
+   Below the Average Price",
+   xlab=NA, ylab=NA, col="blue")
```

Stock Index Weighting Methods

Stock market indices can be either capitalization-weighted, price-weighted, or equal wealth.

The cap-weighted index is equal to the average of the market capitalizations of all its companies (stock price times number of shares). The *S&P500* index is cap-weighted.

The cap-weighted index is equivalent to owning a fixed number of shares, proportional to the number of shares outstanding. So if company X has twice as many shares outstanding as company Y, then the cap-weighted index will own twice as many shares of company X as company Y.

The price-weighted index is equal to the average of the stock prices. The price-weighted index is equivalent to owning a fixed and equal number of shares. The *DJIA* index is price-weighted.

The equal wealth index invests equal dollar amounts in each stock, and it rebalances its weights as market prices change.

The cap-weighted and price-weighted indices are overweight large-cap stocks, compared to the equal wealth index which has larger weights for small-cap stocks.

```
> # Calculate the percentage VTI returns
> retvti <- na.omit(rutils::etfenv$returns$VTI)
> colnames(retvti) <- "VTI"
> datev <- zoo::index(retvti)
> nrow <- NROW(retvti)
> # Load daily S&P500 stock prices
> load(file="/Users/jerzy/Develop/lecture_slides/data/sp500_prices.R")
> # Select the stock prices since VTI
> pricestock <- pricestock[datev]
> # Select stocks with no NA values in their prices
> numna <- sapply(pricestock, function(x) sum(is.na(x)))
> pricestock <- pricestock[, numna == 0]
> # Drop penny stocks
> pricel <- last(pricestock)
> pricel <- drop(coredata(pricel))
> pricestock <- pricestock[, pricel > 1]
> # Calculate the dollar and percentage stock returns
> retld <- rutils::diffit(pricestock)
> retlp <- retld/rutils::lagit(pricestock)
> retlp[1, ] <- 0
> nstocks <- NCOL(retlp)
> # Calculate the returns of equal wealth portfolio
> retew <- rowMeans(retlp, na.rm=TRUE)
> retew[1] <- 0
```

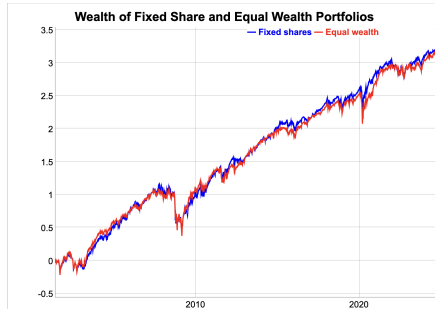
The Equal Wealth Portfolio

The equal wealth portfolio rebalances its weights - it sells the stocks with higher returns and buys stocks with lower returns. So it's a *mean reverting* (contrarian) strategy.

The equal wealth portfolio can often underperform the cap-weighted and fixed share indices because it gradually overweights underperforming stocks, as it rebalances to maintain equal wealth allocations.

In periods when a small number of stocks dominate returns, the cap-weighted and fixed share indices outperform the equal wealth index.

```
> # Wealth of fixed shares portfolio
> wealthfs <- rowMeans(cumprod(1 + retp))
> # Wealth of equal wealth portfolio (with rebalancing)
> wealthew <- cumprod(1 + retew)
```



```
> # Calculate combined log wealth
> wealthv <- cbind(wealthfs, wealthew)
> wealthv <- log(wealthv)
> wealthv <- xts::xts(wealthv, datev)
> colnames(wealthv) <- c("Fixed shares", "Equal wealth")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(rutils::diffit(wealthv), function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of combined log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(wealthv[endw],
+   main="Wealth of Fixed Share and Equal Wealth Portfolios") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Random Stock Selection

A random portfolio is a sub-portfolio of stocks selected at random.

Random portfolios are used as a benchmark for stock pickers (portfolio managers).

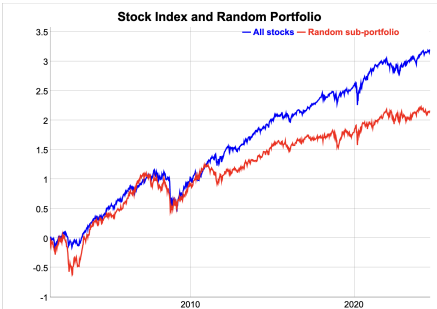
If a portfolio manager outperforms the median of random portfolios, then they may have stock picking skill.

According to S&P Global, **95% of large-cap actively managed funds have underperformed their benchmark.** And the underperformance increases for longer holding periods, because the distribution of stock prices becomes more and more skewed over time.

Charlie Munger (vice chairman of Berkshire Hathaway) said that "Most money managers are little more than fortune tellers or astrologers."

John Bogle (founder of The Vanguard Group) said, "Don't look for the needle in the haystack. Just buy the whole haystack."

```
> # Select a random, fixed share sub-portfolio of 5 stocks
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> samplev <- sample.int(n=nstocks, size=5, replace=FALSE)
> wealthr <- rowMeans(cumprod(1 + retp[, samplev]))
```



```
> # Plot dygraph of index and random sub-portfolio
> wealthv <- cbind(wealthfs, wealthr)
> wealthv <- log(wealthv)
> wealthv <- xts::xts(wealthv, order.by=datev)
> colnames(wealthv) <- c("Index", "Random portfolio")
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(wealthv[endw], main="Stock Index and Random Portfolio")
+ dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+ dyLegend(show="always", width=300)
```

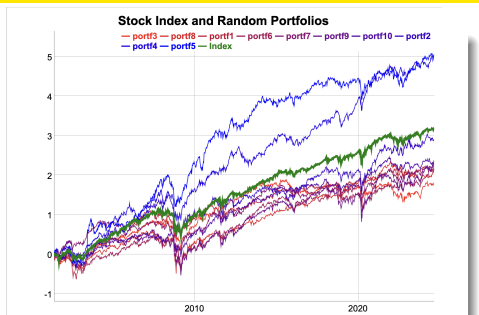
Random Stock Portfolios

Most random portfolios underperform the index, so picking a portfolio which outperforms the stock index requires great skill.

An investor without skill, who selects stocks at random, has a high probability of underperforming the index, because they will most likely miss selecting the best performing stocks.

Therefore the proper benchmark for a stock picker is the median of random portfolios, not the stock index, which is the mean of all the stock prices.

Performing as well as the index requires *significant* investment skill, while outperforming the index requires *exceptional* investment skill.



```
> # Select 10 random fixed share sub-portfolios
> set.seed(1121, "Mersenne-Twister", sample.kind="Rejection")
> nportf <- 10
> wealthr <- sapply(1:nportf, function(x) {
+   samplev <- sample.int(n=nstocks, size=5, replace=FALSE)
+   rowMeans(cumprod(1 + ret[p[, samplev]]))
+ }) ## end sapply
> wealthr <- xts::xts(wealthr, order.by=datev)
> colnames(wealthr) <- paste0("portf", 1:nportf)
> # Sort the sub-portfolios according to performance
> wealthr <- wealthr[, order(wealthr[nrows])]
> round(head(wealthr), 3)
> round(tail(wealthr), 3)
```

```
> # Plot dygraph of all stock index and random sub-portfolios
> colorv <- colorRampPalette(c("red", "blue"))(nportf)
> colorv <- c("green", colorv)
> wealthv <- cbind(wealthfs, wealthr)
> wealthv <- log(wealthv)
> colnames(wealthv)[1] <- "Index"
> symbolv <- colnames(wealthv)
> dygraphs::dygraph(wealthv[endw], main="Stock Index and Random Portfolios")
+ dyOptions(colors=colorv, strokeWidth=1) %>%
+ dySeries(name=symbolv[1], strokeWidth=2) %>%
+ dyLegend(show="always", width=500)
```

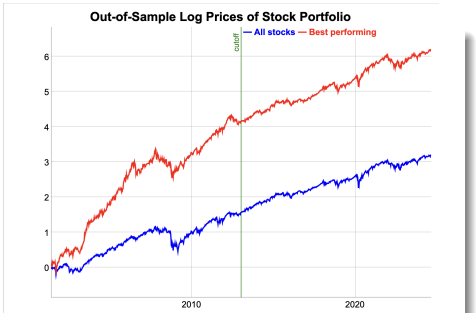
Stock Portfolio Selection Out-of-Sample

The strategy selects the 10 best performing stocks at the end of the in-sample interval, and invests in them in the out-of-sample interval.

The strategy buys equal and fixed number of shares of stocks, and at the end of the in-sample interval, selects the 10 best performing stocks. It then invests the same number of shares in the out-of-sample interval.

The out-of-sample performance of the best performing stocks is not any better than the index.

```
> # Define in-sample and out-of-sample intervals
> cutoff <- nrow %/% 2
> datev[cutoff]
> insample <- 1:cutoff
> outsample <- (cutoff + 1):nrow
> # Calculate the 10 best performing stocks in-sample
> pricev <- cumprod(1 + retsp)
> pricet <- pricev[cutoff, ]
> pricet <- drop(coredata(pricet))
> pricet <- sort(pricet, decreasing=TRUE)
> symbolv <- names(head(pricet, 10))
> # Calculate the wealth of the 10 best performing stocks
> wealthb <- rowMeans(pricev[insample, symbolv])
> wealththos <- wealthb[cutoff]*rowMeans(cumprod(1 + retsp[outsample,
> wealthb <- c(wealthb, wealththos)
```



```
> # Combine the fixed share wealth with the 10 best performing stocks
> wealthv <- cbind(wealthfs, wealthb)
> wealthv <- xts::xts(log(wealthv), order.by=datev)
> colnames(wealthv) <- c("Index", "Best performing")
> # Calculate the in-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(rutils::diffit(wealthv[insample, ]),
+   function(x) c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(rutils::diffit(wealthv[outsample, ]),
+   function(x) c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot out-of-sample stock portfolio returns
> dygraphs::dygraph(wealthv[endw], main="Out-of-Sample Log Prices of
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyEvent(datev[cutoff], label="cutoff", strokePattern="solid", co
+   dyLegend(width=300)
```

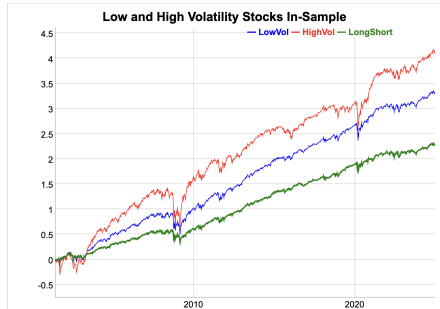
Low and High Volatility Stock Portfolios

Research by Robeco, Eric Falkenstein, and others has shown that low volatility stocks have outperformed high volatility stocks.

Betting against volatility is a strategy which invests in low volatility stocks and shorts high volatility stocks.

USMV is an *ETF* that holds low volatility stocks, although it hasn't met expectations.

```
> # Calculate the stock volatilities, betas, and alphas
> # Perform parallel loop under Mac-OSX or Linux
> library(parallel) ## Load package parallel
> ncores <- detectCores() - 1
> riskret <- mclapply(retp, function(rets) {
+   rets <- na.omit(rets)
+   stdev <- sd(rets)
+   retvti <- retvti[zoo::index(rets)]
+   varvti <- drop(var(retvti))
+   meanvti <- mean(retvti)
+   betac <- drop(cov(rets, retvti))/varvti
+   resid <- rets - betac*retvti
+   alphac <- mean(rets) - betac*meanvti
+   c(alpha=alphac, beta=betac, stdev=stdev, ivol=sd(resid))
+ }, mc.cores=ncores) ## end mclapply
> riskret <- do.call(rbind, riskret)
> tail(riskret)
> # Calculate the median volatility
> riskv <- riskret[, "stdev"]
> medianv <- median(riskv)
> # Calculate the returns of low and high volatility stocks
> retlow <- rowMeans(retp[, (riskv <= medianv)], na.rm=TRUE)
> rethigh <- rowMeans(retp[, (riskv > medianv)], na.rm=TRUE)
```



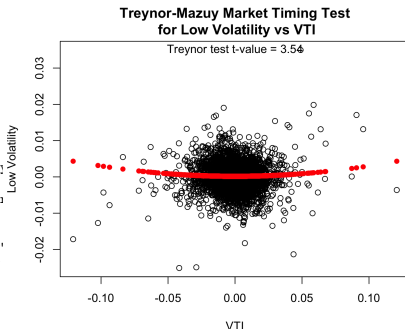
```
> wealthv <- cbind(retlow, rethigh, retlow - 0.25*rethigh)
> wealthv <- xts::xts(wealthv, order.by=datev)
> symbolv <- c("LowVol", "HighVol", "LongShort")
> colnames(wealthv) <- symbolv
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of cumulative returns of low and high volatility stocks
> colorv <- c("blue", "red", "green")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Low and High Volat")
+   dyOptions(colors=colorv, strokeWidth=1) %>%
+   dySeries(name=symbolv[3], strokeWidth=2) %>%
+   dyLegend(width=300)
```

Low Volatility Stock Portfolio Market Timing Skill

Market timing skill is the ability to forecast the direction and magnitude of market returns.

The *Treynor-Mazuy* test shows that the *betting against volatility* strategy has very small *market timing* skill.

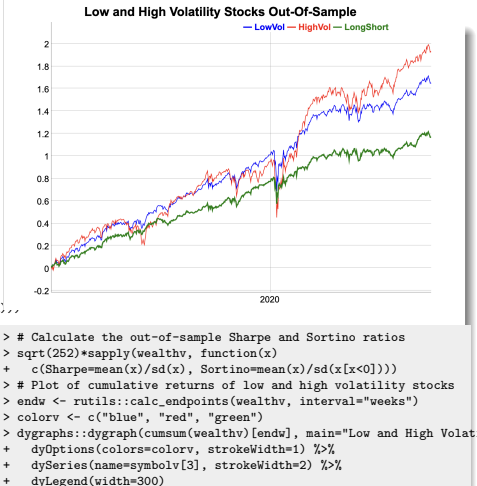
```
> # Merton-Henriksson test
> desm <- cbind(VTI=retvti, 0.5*(retvti+abs(retvti)), retvti^2)
> colnames(desm)[2:3] <- c("Merton", "Treynor")
> regmod <- lm(wealthv$LongShort ~ VTI + Merton, data=desm); summary(regmod)
> # Treynor-Mazuy test
> regmod <- lm(wealthv$LongShort ~ VTI + Treynor, data=desm); summary(regmod)
> # Plot residual scatterplot
> residu <- regmod$residuals
> plot.default(x=retvti, y=residu, xlab="VTI", ylab="Low Volatility")
> title(main="Treynor-Mazuy Market Timing Test\n for Low Volatility")
> # Plot fitted (predicted) response values
> coefreg <- summary(regmod)$coeff
> fitv <- regmod$fitted.values - coefreg["VTI", "Estimate"]*retvti
> tvalue <- round(coefreg["Treynor", "t value"], 2)
> points.default(x=retvti, y=fitv, pch=16, col="red")
> text(x=0.0, y=max(residu), paste("Treynor test t-value =", tvalue))
```



Low and High Volatility Stock Portfolios Out-Of-Sample

The low volatility stocks selected in-sample also have a higher *Sharpe ratio* in the out-of-sample period than the high volatility stocks, although their absolute returns are lower.

```
> # Calculate the in-sample stock volatilities, betas, and alphas
> riskretis <- mclapply(retp[insample], function(rets) {
+   combv <- na.omit(cbind(rets, retvti))
+   if (NROW(combv) > 11) {
+     rets <- na.omit(rets)
+     stdev <- sd(rets)
+     retvti <- retvti[zoo::index(rets)]
+     varvti <- drop(var(retvti))
+     meanvti <- mean(retvti)
+     betac <- drop(cov(rets, retvti))/varvti
+     resid <- rets - betac*retvti
+     alphac <- mean(rets) - betac*meanvti
+     return(c(alpha=alphac, beta=betac, stdev=stdev, ivol=sd(resid),,,
+   } else {
+     return(c(alpha=0, beta=0, stdev=0, ivol=0))
+   } ## end if
+ }, mc.cores=ncores) ## end mclapply
> riskretis <- do.call(rbind, riskretis)
> tail(riskretis)
> # Calculate the median volatility
> riskv <- riskretis[, "stdev"]
> medianv <- median(riskv)
> # Calculate the out-of-sample returns of low and high volatility
> retlow <- rowMeans(retp[outsample, (riskv <= medianv)], na.rm=TRUE)
> rethigh <- rowMeans(retp[outsample, (riskv > medianv)], na.rm=TRUE)
> wealthv <- cbind(retlow, rethigh, retlow - 0.25*rethigh)
> wealthv <- xts::xts(wealthv, order.by=datev[outsample])
> symbolv <- c("LowVol", "HighVol", "LongShort")
> colnames(wealthv) <- symbolv
```

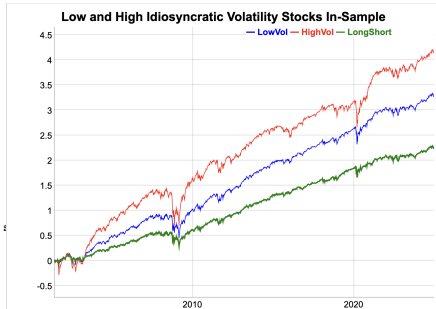


Low and High Idiosyncratic Volatility Stock Portfolios

Research by Robeco, Eric Falkenstein, and others has shown that low idiosyncratic volatility stocks have outperformed high volatility stocks.

Betting against idiosyncratic volatility is a strategy which invests in low idiosyncratic volatility stocks and shorts high volatility stocks.

```
> # Calculate the median idiosyncratic volatility
> riskv <- riskret[, "ivol"]
> medianv <- median(riskv)
> # Calculate the returns of low and high idiosyncratic volatility stocks
> retlow <- rowMeans(retp[, (riskv <= medianv)], na.rm=TRUE)
> rethigh <- rowMeans(retp[, (riskv > medianv)], na.rm=TRUE)
> wealthv <- cbind(retlow, rethigh, retlow - 0.25*rethigh)
> wealthv <- xts::xts(wealthv, order.by=datev)
> symbolv <- c("LowVol", "HighVol", "LongShort")
> colnames(wealthv) <- symbolv
```



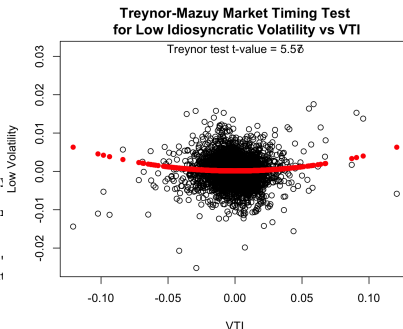
```
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+ c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of returns of low and high idiosyncratic volatility stocks
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Low and High Idiosyncratic Volatility Stocks")
+ dyOptions(colors=colv, strokeWidth=1) %>%
+ dySeries(name=symbolv[3], strokeWidth=2) %>%
+ dyLegend(width=300)
```

Low Idiosyncratic Volatility Stock Portfolio Market Timing Skill

Market timing skill is the ability to forecast the direction and magnitude of market returns.

The *Treynor-Mazuy* test shows that the *betting against idiosyncratic volatility* strategy has some *market timing* skill.

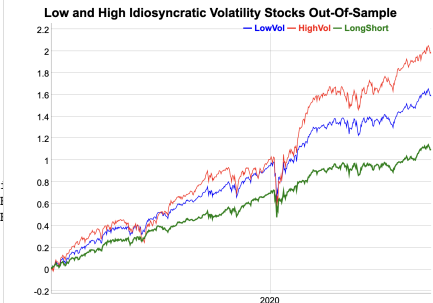
```
> # Merton-Henriksson test
> desm <- cbind(VTI=retvti, 0.5*(retvti+abs(retvti)), retvti^2)
> colnames(desm)[2:3] <- c("Merton", "Treynor")
> regmod <- lm(wealthv$LongShort ~ VTI + Merton, data=desm); summary(regmod)
> # Treynor-Mazuy test
> regmod <- lm(wealthv$LongShort ~ VTI + Treynor, data=desm); summary(regmod)
> # Plot residual scatterplot
> residv <- regmod$residuals
> plot.default(x=retvti, y=residv, xlab="VTI", ylab="Low Volatility")
> title(main="Treynor-Mazuy Market Timing Test\n for Low Idiosyncratic Volatility")
> # Plot fitted (predicted) response values
> coefreg <- summary(regmod)$coeff
> fitv <- regmod$fitted.values - coefreg["VTI", "Estimate"]*retvti
> tvalue <- round(coefreg["Treynor", "t value"], 2)
> points.default(x=retvti, y=fitv, pch=16, col="red")
> text(x=0.0, y=max(residv), paste("Treynor test t-value =", tvalue))
```



Low and High Idiosyncratic Volatility Stock Portfolios Out-Of-Sample

The low idiosyncratic volatility stocks selected in-sample also have a higher *Sharpe ratio* in the out-of-sample period than the high idiosyncratic volatility stocks, although their absolute returns are lower.

```
> # Calculate the median in-sample idiosyncratic volatility
> riskv <- riskretis[, "ivol"]
> medianv <- median(riskv)
> # Calculate the out-of-sample returns of low and high idiosyncratic volatility stocks
> retlow <- rowMeans(retp[outsample, (riskv <= medianv)], na.rm=TRUE)
> rethigh <- rowMeans(retp[outsample, (riskv > medianv)], na.rm=TRUE)
> wealthv <- cbind(retlow, rethigh, retlow - 0.25*rethigh)
> wealthv <- xts::xts(wealthv, order.by=datev[outsample])
> symbolv <- c("LowVol", "HighVol", "LongShort")
> colnames(wealthv) <- symbolv
```



```
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+ c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of out-of-sample returns of low and high volatility stocks
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Low and High Idiosyncratic Volatility Stocks Out-Of-Sample")
+ dyOptions(colors=colorv, strokeWidth=1) %>%
+ dySeries(name=symbolv[3], strokeWidth=2) %>%
+ dyLegend(width=300)
```

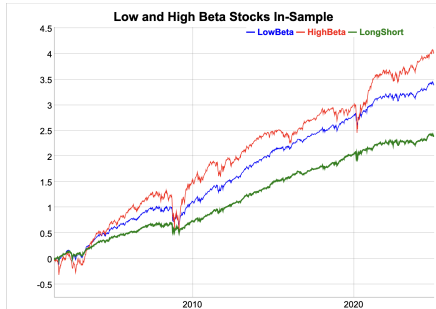
Low and High Beta Stock Portfolios

Research by NYU professors [Andrea Frazzini](#) and [Lasse Heje Pedersen](#) has shown that low beta stocks have outperformed high beta stocks, contrary to the *CAPM* model.

The low beta stocks are mostly from defensive stock sectors, like consumer staples, healthcare, etc., which investors buy when they fear a market selloff.

The strategy of investing in low beta stocks and shorting high beta stocks is known as [betting against beta](#).

```
> # Calculate the median beta
> riskv <- riskret[, "beta"]
> medianv <- median(riskv)
> # Calculate the returns of low and high beta stocks
> betalow <- rowMeans(retp[, names(riskv[riskv <= medianv])], na.rm=TRUE)
> betahigh <- rowMeans(retp[, names(riskv[riskv > medianv])], na.rm=TRUE)
> wealthv <- cbind(betalow, betahigh, betalow - 0.25*betahigh)
> wealthv <- xts::xts(wealthv, order.by=datev)
> symbolv <- c("LowBeta", "HighBeta", "LongShort")
> colnames(wealthv) <- symbolv
```



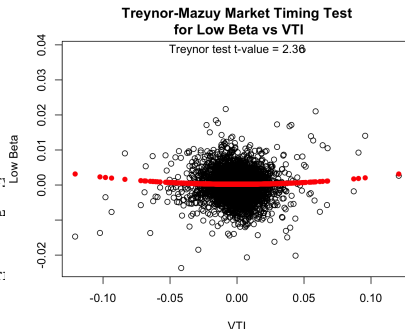
```
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+ c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of cumulative returns of low and high beta stocks
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Low and High Beta Stocks")
+ dyOptions(colors=colorv, strokeWidth=1) %>%
+ dySeries(name=symbolv[3], strokeWidth=2) %>%
+ dyLegend(width=300)
```

Low Beta Stock Portfolio Market Timing Skill

Market timing skill is the ability to forecast the direction and magnitude of market returns.

The *Treynor-Mazuy* test shows that the *betting against beta* strategy does not have significant *market timing* skill.

```
> # Merton-Henriksson test
> desm <- cbind(VTI=retvti, 0.5*(retvti+abs(retvti)), retvti^2)
> colnames(desm)[2:3] <- c("Merton", "Treynor")
> regmod <- lm(wealthv$LongShort ~ VTI + Merton, data=desm); summary
> # Treynor-Mazuy test
> regmod <- lm(wealthv$LongShort ~ VTI + Treynor, data=desm); summary
> # Plot residual scatterplot
> residv <- regmod$residuals
> plot.default(x=retvti, y=residv, xlab="VTI", ylab="Low Beta")
> title(main="Treynor-Mazuy Market Timing Test\n for Low Beta vs VTI")
> # Plot fitted (predicted) response values
> coefreg <- summary(regmod)$coeff
> fitv <- regmod$fitted.values - coefreg["VTI", "Estimate"]*retvti
> tvalue <- round(coefreg["Treynor", "t value"], 2)
> points.default(x=retvti, y=fitv, pch=16, col="red")
> text(x=0.0, y=max(residv), paste("Treynor test t-value =", tvalue))
```



Low and High Beta Stock Portfolios Out-Of-Sample

The low beta stocks selected in-sample also have a higher *Sharpe ratio* in the out-of-sample period than the high beta stocks, although their absolute returns are lower.

```
> # Calculate the median beta
> riskv <- riskretis[, "beta"]
> medianv <- median(riskv)
> # Calculate the out-of-sample returns of low and high beta stocks
> betalow <- rowMeans(retp[outsample, names(riskv[riskv <= medianv])])
> betahigh <- rowMeans(retp[outsample, names(riskv[riskv > medianv])])
> wealthv <- cbind(betalow, betahigh, betalow - 0.25*betahigh)
> wealthv <- xts::xts(wealthv, order.by=datev[outsample])
> symbolv <- c("LowBeta", "HighBeta", "LongShort")
> colnames(wealthv) <- symbolv
```



```
> # Calculate the out-of-sample Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+ c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of out-of-sample returns of low and high beta stocks
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw], main="Low and High Beta S
+ dyOptions(colors=colorv, strokeWidth=1) %>%
+ dySeries(name=symbolv[3], strokeWidth=2) %>%
+ dyLegend(width=300)
```

Stocks With Low and High Trailing Volatilities

The trailing volatilities can be used to create low and high volatility portfolios, and test their performance out-of-sample.

The low volatility portfolio consists of stocks with trailing volatilities less than the median, and the high portfolio with trailing volatilities greater than the median.

The portfolios are rebalanced daily, as the volatility changes.

The low volatility portfolio has a higher *Sharpe ratio*, but lower absolute returns than the high volatility portfolio.

```
> # Calculate the trailing percentage volatilities
> volp <- HighFreq::run_var(retp, lambda=0.15)
> volp <- sqrt(volp[, (nstocks+1):(2*nstocks)])
> volp <- rutils::lagit(volp)
> volp[volp == 0] <- 1
> # Calculate the median volatilities
> medianv <- matrixStats::rowMedians(volp)
> # Calculate the wealth of low volatility stocks
> weightv <- (volp <= medianv)
> weightv <- rutils::lagit(weightv)
> retlow <- rowMeans(weightv*retp)
> # Calculate the wealth of high volatility stocks
> weightv <- (volp > medianv)
> weightv <- rutils::lagit(weightv)
> rethigh <- rowMeans(weightv*retp)
```



```
> # Combined wealth
> wealthv <- cbind(retlow, rethigh)
> wealthv <- xts::xts(wealthv, datev)
> colnames(wealthv) <- c("LowVol", "HighVol")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Wealth of Low and High Volatility Stocks") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

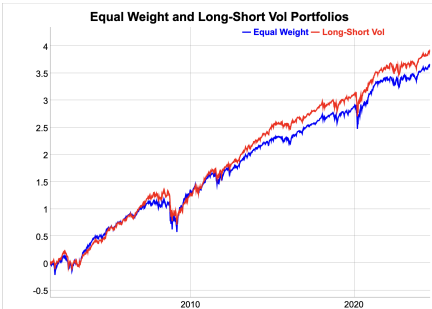

Long-Short Stock Volatility Strategy

The *Long-Short Volatility* strategy buys the low volatility stock portfolio and shorts the high volatility portfolio.

The high volatility portfolio returns are multiplied by a factor to compensate for their higher volatility.

The *Long-Short Volatility* strategy has a higher *Sharpe ratio* and also higher absolute returns than the equal wealth strategy.

```
> # Calculate the long-short volatility returns
> retls <- (retlow - 0.25*rethigh)
> # Scale the PnL volatility to that of wealthw
> retls <- retls*sd(retew)/sd(retls)
> # Combined wealth
> wealthv <- cbind(retew, retls)
> wealthv <- xts::xts(wealthv, datev)
> symbolv <- c("EqualWeight", "Long-Short Vol")
> colnames(wealthv) <- symbolv
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(wealthv, function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0])))
```



```
> # Plot of log wealth
> dygraphs::dygraph(cumsum(wealthv)[endw],
+   main="Equal Weight and Long-Short Vol Portfolios") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=300)
```

Risk Parity Strategy Weights

The dollar amount of a stock with price p_i that has unit dollar volatility is equal to: $\frac{p_i}{\sigma_i^d}$.

Where σ_i^d is the dollar volatility.

So the weights of the risk parity strategy are proportional to the inverse of the stock dollar volatilities: $w_i \propto \frac{1}{\sigma_i^d}$.

The weights are rebalanced daily so that the dollar volatilities of the stock allocations (*dollar amounts*) remain equal.

The stock allocations increase when the volatility is low, and vice versa.

The function `HighFreq::run_var()` calculates the trailing variance of a *time series* of returns r_t , by recursively weighting the past variance estimates σ_{t-1}^2 , with the squared differences of the returns minus the trailing means $(r_t - \bar{r}_t)^2$, using the decay factor λ :

$$\bar{r}_t = \lambda \bar{r}_{t-1} + (1 - \lambda) r_t$$

$$\sigma_t^2 = \lambda^2 \sigma_{t-1}^2 + (1 - \lambda^2) (r_t - \bar{r}_t)^2$$

Where σ_t^2 is the trailing variance at time t .

The decay factor λ determines how quickly the variance estimates are updated, with smaller values of λ producing faster updating, giving more weight to recent returns, and vice versa.

```
> # Calculate the trailing dollar volatilities
> lambdav <- 0.99
> vold <- HighFreq::run_var(retsd, lambda=lambdav)
> vold <- vold[, (nstocks+1):(2*nstocks)]
>
> vold <- sqrt(vold)
> vold[vold == 0] <- 1
> # Calculate the rolling risk parity weights
> weightv <- 1/vold
> weightv <- weightv/rowSums(weightv)
```

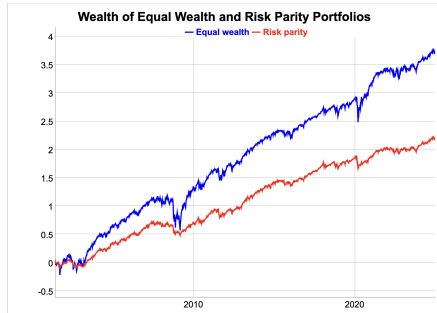
Risk Parity Strategy for Stocks

The risk parity strategy for stocks has a similar *Sharpe ratio* to the equal wealth strategy. But it has lower absolute returns.

The risk parity strategy also has higher transaction costs.

Risk parity for stocks doesn't perform as well as for assets with negative or low correlations, like stocks and bonds, because stock returns have positive correlations.

```
> # Calculate the risk parity allocations
> pricerp <- pricestock*weightv
> # Calculate the dollar returns of risk parity
> retrp <- retrp*rutils::lagit(pricerp)
> # Calculate the wealth of risk parity
> retrp <- retrp*sd(retew)/sd(retrp)
> wealthrp <- cumsum(rowMeans(retrp))
> # Wealth of equal wealth portfolio (with rebalancing)
> wealthew <- cumsum(retew)
```



```
> # Combined wealth
> wealthv <- cbind(wealthew, wealthrp)
> wealthv <- xts::xts(wealthv, datev)
> colnames(wealthv) <- c("Equal wealth", "Risk parity")
> # Calculate the Sharpe and Sortino ratios
> sqrt(252)*sapply(rutils::diffit(wealthv), function(x)
+   c(Sharpe=mean(x)/sd(x), Sortino=mean(x)/sd(x[x<0]))))
> # Plot of log wealth
> endw <- rutils::calc_endpoints(wealthv, interval="weeks")
> dygraphs::dygraph(wealthv[endw],
+   main="Wealth of Equal Wealth and Risk Parity Portfolios") %>%
+   dyOptions(colors=c("blue", "red"), strokeWidth=2) %>%
+   dyLegend(show="always", width=400)
```

Homework Assignment

Required

- Study all the lecture slides in *FRE7241_Lecture_4.pdf*, and run all the code in *FRE7241_Lecture_4.R*

Recommended

- Download from NYU Classes and read about momentum strategies:
Moskowitz Time Series Momentum.pdf
Bouchaud Momentum Mean Reversion Equity Returns.pdf
Hurst Pedersen AQR Momentum Evidence.pdf